AN INTEGRATED TOPSIS FRAMEWORK WITH FULL-RANGE WEIGHT SENSITIVITY ANALYSIS FOR ROBUST DECISION ANALYSIS

UAB
Universitat Autònoma de Barcelona

Research Group on Biological Treatment and Valorisation of Liquid and Gaseous Effluents
GENOCOV

Àlex Gaona Soler, Albert Guisasola and Juan Antonio Baeza

Departament d'Enginyeria Química, Biològica i Ambiental
Escola d'Enginyeria. Universitat Autònoma de Barcelona

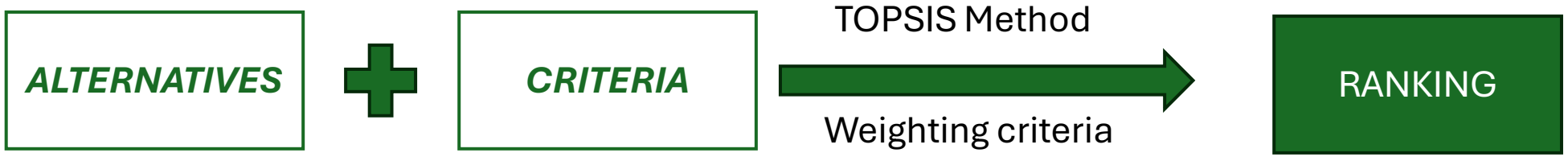
Email: alex.gaona@uab.cat, albert.guisasola@uab.cat, juanantonio.baeza@uab.cat

1. INTRODUCTION

- **TOPSIS** (Technique for Order of Preference by Similarity to Ideal Solution)

  → multi-criteria decision method for selecting the best alternative among a set of options.

  - It is based on the idea that the ideal alternative is the one that is closest to the positive-ideal solution and furthest from the negative-ideal solution.



- Sensitivity analysis is a useful tool for assessing the robustness of TOPSIS results to changes in the weighting of criteria.
- The aim of this work is:
  - Show users how the TOPSIS method can be applied using Matlab.
  - To teach the user the importance of weighting selection in a case study, applying two sensitivity analysis methods to evaluate the impact of changes in weights on the ranking of alternatives.

2. TOPSIS METHOD

Step 1 - Selection of criteria, weights, and MaxMin vector

- These criteria must be **relevant** to the decision-making problem and must be **quantifiable**.
  The decision matrix (*C*) for *m* alternatives and n criteria is then built as *equation (1)*.

Matrix (C) dimensions

Rows = *m* Alternatives (*i*)
Columns = *n* Criteria (*j*)

$$Decision\ matrix\ (C) = \begin{array}{c} \begin{array}{ccccc} \text{Crit. 1} & \text{Crit. 2} & \text{Crit. } j & \cdots & \text{Crit. } n \end{array} \\ \begin{bmatrix} c_{11} & c_{12} & c_{1j} & \cdots & c_{1n} \\ c_{21} & c_{22} & c_{2j} & \cdots & c_{2n} \\ c_{i1} & c_{i2} & c_{ij} & \cdots & c_{in} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ c_{m1} & c_{m2} & c_{mj} & \cdots & c_{mn} \end{bmatrix} \end{array} \begin{array}{l} \leftarrow \text{Alt. 1} \\ \leftarrow \text{Alt. 2} \\ \leftarrow \text{Alt. } i \\ \vdots \\ \leftarrow \text{Alt. } m \end{array} \qquad (1)$$

- c_{ij}, *i* represents the **alternative**, while *j* refers to the **criterion**
- Select the weights of each criterion (value of each weight between 0 and 1 and sum of all weights must be 1).

$$W = [\omega_1, \omega_2, \omega_3, \cdots \omega_j]$$

- Select MaxMin values. MaxMin values are either +1 or -1.
  +1 represents a criterion to be maximised (**a higher value is preferred in the ideal alternative**)
  -1 represents a criterion to be minimised (**a lower value is preferred in the ideal alternative**).

$$MM = [MM_1, MM_2, MM_3, \cdots\ MM_j]$$

Step 2 - Normalization of the decision matrix (C)

- The criteria may have different units or scales of measurement, which can make it difficult to compare alternatives. Then, the criteria values are normalised to convert them to a similar scale.

Matrix (*R*) dimensions

Rows = *m* Alternatives (*i*)
Columns = *n* Criteria (*j*)

$$Stand.\,decision\ matrix\ (R) = \begin{bmatrix} r_{11} & r_{12} & r_{1j} & \cdots & r_{1n} \\ r_{21} & r_{22} & r_{2j} & \cdots & r_{2n} \\ r_{i1} & r_{i2} & r_{ij} & \cdots & r_{in} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ r_{m1} & r_{m2} & r_{mj} & \cdots & r_{mn} \end{bmatrix} \qquad (2)$$

- Where r_{ij} is calculated with *equation (3)* by normalizing the value of criterion c_{ij} and using the sign selected in the *MaxMin* vector:

$$r_{ij} = \frac{c_{ij}}{\left[\sum_{i=1}^{m} c_{ij}\right]^{1/2}} \cdot MM_j \qquad (3)$$

Step 3 - Matrix weighting

- This step involves using the weights defined in Step 1 to ensure that each criterion has the desired importance in the evaluation of alternatives.

Matrix (*F*) dimensions

Rows = *m* Alternatives (*i*)
Columns = *n* Criteria (*j*)

$$Weighted\ normalized\ matrix\ (F) = \begin{array}{c} \begin{array}{ccccc} \text{Crit. 1} & \text{Crit. 2} & \text{Crit. } j & \cdots & \text{Crit. } n \end{array} \\ \begin{array}{ccccc} \omega_1 & \omega_2 & \omega_j & \cdots & \omega_n \end{array} \\ \begin{bmatrix} f_{11} & f_{12} & f_{1j} & \cdots & f_{1n} \\ f_{21} & f_{22} & f_{2j} & \cdots & f_{2n} \\ f_{i1} & f_{i2} & f_{ij} & \cdots & f_{in} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ f_{m1} & f_{m2} & f_{mj} & \cdots & f_{mn} \end{bmatrix} \end{array} \quad (4)$$

- Where f_{ij} is the weighted normalized criterion value calculated with *equation (5)* with ω_j being the weight assigned to each criterion (values in the *W* vector).

$$f_{ij} = r_{ij} \cdot \omega_j \quad (5)$$

Step 4 - Determination of positive and negative ideal solutions

- The positive and negative-ideal values for each criterion can be defined with *equation (6)*.

$$\begin{cases} f_j^+ = max\{f_{1j}, f_{2j}, f_{3j}, \dots f_{mj}\} \\ f_j^- = min\{f_{1j}, f_{2j}, f_{3j}, \dots f_{mj}\} \end{cases} \quad (j = 1, 2, 3, \dots n) \qquad (6)$$

- The positive-ideal solution (***PIS***) (*equation 6*) includes the positive ideal value for each criterion.
- The negative-ideal solution (***NIS***) (*equation 7*) includes the negative ideal value for each criterion.

$$PIS = \{f_1^+, f_2^+, f_3^+, \dots f_n^+\} \qquad (7)$$

$$NIS = \{f_1^-, f_2^-, f_3^-, \dots f_n^-\} \qquad (8)$$

Step 5 - Distance calculation to the positive- and negative-ideal solutions

- This step assesses the relative proximity of each alternative *i* to *PIS* (d_i^+) and *NIS* (d_i^-), *equation (9)*.

$$\begin{cases} d_i^+ = \sqrt{\sum_{j=1}^{n} \left(f_{ij} - f_j^+\right)^2} \\ d_i^- = \sqrt{\sum_{j=1}^{n} \left(f_{ij} - f_j^-\right)^2} \end{cases} \qquad (9)$$

Step 6 - Distance calculation to the positive- and negative-ideal solutions

- CC_i (equation 10) shows the closeness coefficients, how far an alternative *i* is from *NIS*.

$$Closeness\ Coefficient\ (CC_i) = {d_i^-}/{(d_i^+ + d_i^-)} \qquad (10)$$

- ↓ CC_i the alternative *i* is closer to *NIS* and further away from *PIS* (bad alternative).

- ↑ CC_i the alternative *i* is closer to *PIS* and further away from *NIS* (good alternative).

- Therefore, the TOPSIS ranking is established in order of CC_i values from highest to lowest, resulting in a rank of alternatives ordered according to their distance to the negative ideal solution, and equivalently to its proximity to the positive ideal solution.

CC (Rank 1) > *CC* (Rank 2) > *CC* (Rank 3) > … > *CC* (Rank n) → **RANKING**

Rank *1*
Rank *2*
Rank *3*
…
Rank *n*

User manual:

- TOPSIS was implemented in MATLAB® following the mathematical equations detailed in (Li et al., 2013).

- This section explains how each step of the TOPSIS method is programmed (*fun_TOPSIS*), and the steps the user must follow to use the TOPSIS method in a case study (*main_TOPSIS*).

main_TOPSIS	*fun_TOPSIS*
The user must perform **4 steps:** 1. Define the matrix C 2. Select the value of each of the weights, vector W 3. Define the vector MaxMin 4. Call the function *fun_TOPSIS*	This MATLAB file contains all the steps of the TOPSIS method (Step 1-6) as explained previously.

User manual (main_TOPSIS_cars):

1. Define the decision matrix (C):

- The user must write the decision matrix C.
- c_{ij}, where *i* represents the **alternative**, while *j* refers to the values associated with each **criterion** for this alternative.
- Number of rows = *m* Alternatives
- Number of columns = *n* Criteria.

user manual

```
%% MAIN TOPSIS
%% CONSTRUCTION OF THE INITIAL DECISION MATRIX (C)
% DECISION MATRIX (C)

%  Crit 1   Crit 2   Crit 3  Crit 4  Crit 5  Crit 6   Crit 7
C = [500      180      306      1     51750    7         2;   % Alternative 1
     510      180      110     4.8    24540    5         5;   % Alternative 2
     470      160      220      0     43488    5         1;   % Alternative 3
     395      196      120     5.5    29800    4         4;   % Alternative 4
     377      170      122     5.3    29700    5         3];  % Alternative 5
```

User manual (main_TOPSIS):

2. Select the value of each weight (W):

- The user must select the weights for each of the criteria studied.
- Each position of the vector W represents the weight chosen for each criterion.

```
%% Weight vector (W) - Relative importance of each criterion
wi = 1 / 7; %The seven criteria the same weight
W = [wi wi wi wi wi wi wi];
```

3. Define the MaxMin vector:

- The values of MaxMin (MM) are either +1 or -1.

+1 represents a criterion to be maximised (**a higher value is preferred in the ideal alternative**).
-1 represents a criterion to be minimised (**a lower value is preferred in the ideal alternative**).

```
%% MaxMin vector (MM) - Direction of impact for each criterion (1 to maximize, -1 to minimize)
% In other words, here it is important if it is more favorable that the studied value is larger (1) or smaller (-1).
MM = [1 1 1 -1 -1 1 -1];
```

User manual (main_TOPSIS):

4. Call the fun_TOPSIS function

- This is the last step, where the function fun_TOPSIS is called (code details explained below).
- This function has as inputs,
  - the decision matrix (*C*)
  - the vector of weights (*W*)
  - the MaxMin vector (*MM*), previously defined

INPUTS

- and has as outputs,
  - Ranking, which gives us the ranking from the best alternative to the last alternative
  - CC values, the closeness coefficient
  - d^+ and d^- values, the distance to the positive- and negative-ideal solutions (*PIS* and *NIS*)

OUTPUTS

```
[ranking,CC,D_positive,D_negative] = fun_TOPSIS(C,W,MM);
```

User manual (main_TOPSIS):

```
%% MAIN TOPSIS + WSA + FRWSA
%% CONSTRUCTION OF THE INITIAL DECISION MATRIX (C)
% DECISION MATRIX (C)

% Comparison of five cars and seven criteria
 %Trunk(L) MaxSp(km/h) Power(hp) L/100km Cost  N.Seats  Combust.
C = [500      180       306      1      51750    7         2;   % CAR 1
     510      180       110     4.8     24540    5         5;   % CAR 2
     470      160       220      0      43488    5         1;   % CAR 3
     395      196       120     5.5     29800    4         4;   % CAR 4
     377      170       122     5.3     29700    5         3];  % CAR 5

%% Weight vector (W) - Relative importance of each criterion
[N_Alt,N_Crit] = size(C);
W = ones(1,N_Crit)/N_Crit; %Same weight for all the criteria
% wi = 1 / 7; %
% W = [wi wi wi wi wi wi wi]; Other way to define the W vector

%% MaxMin vector (MM) - Direction of impact for each criterion (1 to maximize, -1 to minimize)
% In other words, here it is important if it is more favorable that the studied value is larger (1) or smaller (-1).
MM = [1 1 1 -1 -1 1 -1];

%% TOPSIS calculation
[ranking,CC,D_positive,D_negative] = fun_TOPSIS(C,W,MM)
figure(1)
[N_Alt,~] = size(C);
bar(CC)
axis([0 N_Alt+1 0 1]);
xlabel('Alternative')
ylabel('Closeness coefficient')
title('TOPSIS ranking (higher better)')
figure(2)
bar(D_positive,'red')
xlabel('Alternative')
ylabel('Distance to best solution')
title('TOPSIS Alternative distance to PIS (lower better)')
figure(3)
bar(D_negative,'cyan')
xlabel('Alternative')
ylabel('Distance to worst solution')
title('TOPSIS Alternative distance to NIS (higher better)')
```

User manual (fun_TOPSIS):

1. *Step 1* – Determination of criteria, weights, and MaxMin vector.

- In this step of the MATLAB code, given the decision matrix (C), the weights and MaxMin values are checked to ensure they correspond to the number of criteria to be studied.

2. *Step 2 – Normalisation of the decision matrix (C).*

- Matrix R size is initialized with the code *R = **zeros**(m, n);*
- The values are then normalized, *equations (2) – (3)*,
  - *R = C ./ **vecnorm**(C). Vecnorm* is a MATLAB function that returns the Euclidean norm of each column of the matrix C.
- with the input of the vector MaxMin (*MM*), the values of the matrix R are changed in sign if the specific criterion needs to be minimised.

```
function [ranking,CC,D_positive,D_negative] = fun_TOPSIS(C,W,MM)
%% STEP 1: RECEPTION OF CRITERIA, WEIGHTS, AND MAXMIN VECTOR.
[m, n] = size(C);

%To ensure that the user has set as inputs as many weights as there are criteria in the study, and the MaxMin vector has the same size as the criteria.
if length(W) ~= n || length(MM) ~= n

else
    disp('The weights (W) and the MaxMin vector do not correspond to the number of criteria to be studied.')
end

% m: number of alternatives to evaluate; n: number of criteria;
% C(i,j): for the alternative i, value for criterion j

%% STEP 2: NORMALIZATION OF THE DECISION MATRIX (C)
R = zeros(m, n); %#ok<PREALL>
R = C ./ vecnorm(C); %Calculates R matrix of equations (2)-(3)

% We change sign if the criterion is of the type minimization
R(:, MM ~= 1) = -R(:, MM ~= 1);
```

User manual (fun_TOPSIS):

3. Step 3 – Matrix weighting.

- The normalised values of the matrix R are multiplied by the corresponding *W* weights assigned to each criterion.

4. Step 4 – Determination of positive and negative ideal solutions.

- The positive ideal solution is calculated by selecting the best value for each criterion, while the negative ideal solution is obtained by selecting the worst value for each criterion (MATLAB functions *max* and *min).*

```
%% STEP 3: Matrix weighting
%The weighted normalized matrix (F) is a matrix representing the weighted normalized criterion values of each alternative
F = R .* W;  %Calculates F matrix according to equiations (4) and (5)

%% STEP 4: DETERMINATION OF G= + IDEAL SOLUTION & B= - IDEAL SOLUTION
PIS = max(F); % According to equations (6) and (7)
NIS = min(F); % According to equations (6) and (8)
```

User manual (fun_TOPSIS):

5. Step 5 – Distance calculation to the positive- and negative-ideal solutions.

- The distances of each alternative to the ideal positive and ideal negative solutions are calculated.

6. Step 6 – TOPSIS ranking.

- The closeness coefficient (CC) is calculated, *CC = D_negative ./ (D_positive + D_negative).*
- The CC are then sorted from highest to lowest using the *sort* MATLAB function, resulting in the final ranking of the alternatives.

[~, ranking] = sort(CC, 'descend').

```
%% STEP 5: CALCULATION OF THE DISTANCES TO THE + & - IDEAL SOLUTIONS
D_positive = sqrt(sum((F - PIS).^2, 2)); % According to equation (9+)
D_negative = sqrt(sum((F - NIS).^2, 2)); % According to equation (9-)

%% STEP 6: CALCULATION OF THE CLOSENESS COEFICIENT (CC) OF EACH ALTERNATIVE AND TOPSIS RANKING
CC = D_negative ./ (D_positive + D_negative); % According to equation (10)
[~, ranking] = sort(CC, 'descend'); % Ordering CC results for TOPSIS rank

end
```

User manual (fun_TOPSIS):

```
function [ranking,CC,D_positive,D_negative] = fun_TOPSIS(C,W,MM)
%% STEP 1: RECEPTION OF CRITERIA, WEIGHTS, AND MAXMIN VECTOR.
[m, n] = size(C);

%To ensure that the user has set as inputs as many weights as there are criteria in the study, and the MaxMin vector has the same size as the criteria.
if length(W) ~= n || length(MM) ~= n

else
    disp('The weights (W) and the MaxMin vector do not correspond to the number of criteria to be studied.')
end

% m: number of alternatives to evaluate; n: number of criteria;
% C(i,j): for the alternative i, value for criterion j

%% STEP 2: NORMALIZATION OF THE DECISION MATRIX (C)
R = zeros(m, n); %#ok<PREALL>
R = C ./ vecnorm(C); %Calculates R matrix of equations (2)-(3)

% We change sign if the criterion is of the type minimization
R(:, MM ~= 1) = -R(:, MM ~= 1);

%% STEP 3: Matrix weighting
%The weighted normalized matrix (F) is a matrix representing the weighted normalized criterion values of each alternative
F = R .* W;  %Calculates F matrix according to equiations (4) and (5)

%% STEP 4: DETERMINATION OF G= + IDEAL SOLUTION & B= - IDEAL SOLUTION
PIS = max(F); % According to equations (6) and (7)
NIS = min(F); % According to equations (6) and (8)

%% STEP 5: CALCULATION OF THE DISTANCES TO THE + & - IDEAL SOLUTIONS
D_positive = sqrt(sum((F - PIS).^2, 2)); % According to equation (9+)
D_negative = sqrt(sum((F - NIS).^2, 2)); % According to equation (9-)

%% STEP 6: CALCULATION OF THE CLOSENESS COEFICIENT (CC) OF EACH ALTERNATIVE AND TOPSIS RANKING
CC = D_negative ./ (D_positive + D_negative); % According to equation (10)
[~, ranking] = sort(CC, 'descend'); % Ordering CC results for TOPSIS rank
end
```

3.TOPSIS METHOD. CASE STUDY

TOPSIS METHOD. CASE STUDY.

- The following case study is presented to facilitate a better understanding of the TOPSIS method.
- The objective is to identify the optimal car among a selection of five.
- The following criteria were considered in the analysis: **Trunk** capacity (L), **maximum speed** (km/h), **power** of the car (hp), **consumption** per 100 km (L/100km), **cost** of the car (€), **number of seats** and the **type of fuel** used (1: Electric; 2: Plug-In Hybrid; 3:Hybrid; 4: Gasoline; 5: Diesel).

CAR 1

- **Trunk:** 500 L
- **MaxSp:** 180 km/h
- **Power:** 306 hp
- **L/100km:** 1 L/100km
- **Cost :** 51,750 €
- **N. Seats:** 7
- **Fuel :** 2

CAR 2

- **Trunk:** 510 L
- **MaxSp:** 180 km/h
- **Power:** 110 hp
- **L/100km:** 4.8 L/100km
- **Cost :** 24,540 €
- **N. Seats:** 5
- **Fuel :** 5

CAR 3

- **Trunk:** 470 L
- **MaxSp:** 160 km/h
- **Power:** 220 hp
- **L/100km:** 0 L/100km
- **Cost :** 43,488 €
- **N. Seats:** 5
- **Fuel :** 1

CAR 4

- **Trunk:** 395 L
- **MaxSp:** 196 km/h
- **Power:** 120 hp
- **L/100km:** 5.5 L/100km
- **Cost :** 29,800 €
- **N. Seats:** 4
- **Fuel :** 4

CAR 5

- **Trunk:** 377 L
- **MaxSp:** 170 km/h
- **Power:** 122 hp
- **L/100km:** 5.3 L/100km
- **Cost :** 29,700€
- **N. Seats:** 5
- **Fuel :** 3

Step 1 - Selection of criteria, weights, and MaxMin vector.

- In this case there are **5 alternatives**, and **7 criteria** to be studied, so the decision matrix (C) would be as follows:

Matrix (C) dimensions

Rows = *5* Alternatives
Columns = *7* Criteria

$$Decision\ matrix\ (C) = \begin{array}{c} \begin{array}{ccccccc} \textit{Trunk} & \textit{Max.Sp} & \textit{Power} & \textit{L/100} & \textit{Cost} & \textit{Seats} & \textit{Fuel} \end{array} \\ \begin{bmatrix} 500 & 180 & 306 & 1 & 51750 & 7 & 2 \\ 510 & 180 & 110 & 4.8 & 24540 & 5 & 5 \\ 470 & 160 & 220 & 0 & 43488 & 5 & 1 \\ 395 & 196 & 120 & 5.5 & 29800 & 4 & 4 \\ 377 & 170 & 122 & 5.3 & 29700 & 5 & 3 \end{bmatrix} \end{array} \begin{array}{l} \\ \textit{Car 1} \\ \textit{Car 2} \\ \textit{Car 3} \\ \textit{Car 4} \\ \textit{Car 5} \end{array}$$

- For this study, all weights were assigned the same value (**7 weights**).

$$W = [\frac{1}{7}, \frac{1}{7}, \frac{1}{7}, \frac{1}{7}, \frac{1}{7}, \frac{1}{7}, \frac{1}{7}]$$

- MaxMin vector (**7 MaxMin values**)

$$MM = [1, 1, 1, -1, -1, 1, -1]$$

  - In this case, **Trunk**, **Max. Sp.**, **Power**, **N. Seats**, should have the highest possible value.
  - **L/100km**, **Cost** and **Fuel**, should be minimum.

Step 2 - Normalization of the decision matrix (C).

- To calculate the matrix with the normalised values of the decision matrix (*C*), *equation (3a)* is used, giving as a result:

Equation 3a

$$\boxed{r_{ij} = \frac{c_{ij}}{\left[\sum_{i=1}^{m} c_{ij}\right]^{1/2}}}$$

$$Stand.\,decision\ matrix\ (R) = \begin{array}{c} \begin{array}{ccccccc} \textit{Trunk} & \textit{Max. Sp} & \textit{Power} & \textit{L/100} & \textit{Cost} & \textit{Seats} & \textit{Fuel} \end{array} \\ \begin{bmatrix} 0.4929 & 0.4532 & 0.7145 & 0.1102 & 0.6211 & 0.5916 & 0.2697 \\ 0.5027 & 0.4532 & 0.2568 & 0.5288 & 0.2945 & 0.4226 & 0.6742 \\ 0.4633 & 0.4029 & 0.5137 & 0.0000 & 0.5220 & 0.4226 & 0.1348 \\ 0.3894 & 0.4935 & 0.2802 & 0.6060 & 0.3577 & 0.3381 & 0.5394 \\ 0.3716 & 0.4381 & 0.2849 & 0.5839 & 0.3565 & 0.4226 & 0.4045 \end{bmatrix} \end{array} \begin{array}{l} \\ \textit{Car 1} \\ \textit{Car 2} \\ \textit{Car 3} \\ \textit{Car 4} \\ \textit{Car 5} \end{array}$$

- With the MaxMin vector (*MM*), the sign of r_{ij} is changed.

$$MM = [1\,;\ 1\,;\ 1\,; -1\,; -1\,;\ 1\,; -1]$$

Step 2 - Normalization of the decision matrix (C).

Sign change

$$\boxed{r_{ij} = r_{ij} \cdot MM_j}$$

Equation 3b

$$MM = [1;\ 1;\ 1;\ -1;\ -1;\ 1;\ -1]$$

$$Stand.\,decision\ matrix\ (R) = \begin{array}{c} \begin{array}{ccccccc} \text{Trunk} & \text{Max. Sp} & \text{Power} & \text{L/100} & \text{Cost} & \text{Seats} & \text{Fuel} \end{array} \\ \begin{bmatrix} 0.4929 & 0.4532 & 0.7145 & 0.1102 & 0.6211 & 0.5916 & 0.2697 \\ 0.5027 & 0.4532 & 0.2568 & 0.5288 & 0.2945 & 0.4226 & 0.6742 \\ 0.4633 & 0.4029 & 0.5137 & 0.0000 & 0.5220 & 0.4226 & 0.1348 \\ 0.3894 & 0.4935 & 0.2802 & 0.6060 & 0.3577 & 0.3381 & 0.5394 \\ 0.3716 & 0.4381 & 0.2849 & 0.5839 & 0.3565 & 0.4226 & 0.4045 \end{bmatrix} \end{array} \begin{array}{c} \\ \text{Car 1} \\ \text{Car 2} \\ \text{Car 3} \\ \text{Car 4} \\ \text{Car 5} \end{array}$$

$$Stand.\,decision\ matrix\ (R) = \begin{array}{c} \begin{array}{ccccccc} \text{Trunk} & \text{Max. Sp} & \text{Power} & \text{L/100} & \text{Cost} & \text{Seats} & \text{Fuel} \end{array} \\ \begin{bmatrix} 0.4929 & 0.4532 & 0.7145 & -0.1102 & -0.6211 & 0.5916 & -0.2697 \\ 0.5027 & 0.4532 & 0.2568 & -0.5288 & -0.2945 & 0.4226 & -0.6742 \\ 0.4633 & 0.4029 & 0.5137 & -0.0000 & -0.5220 & 0.4226 & -0.1348 \\ 0.3894 & 0.4935 & 0.2802 & -0.6060 & -0.3577 & 0.3381 & -0.5394 \\ 0.3716 & 0.4381 & 0.2849 & -0.5839 & -0.3565 & 0.4226 & -0.4045 \end{bmatrix} \end{array} \begin{array}{c} \\ \text{Car 1} \\ \text{Car 2} \\ \text{Car 3} \\ \text{Car 4} \\ \text{Car 5} \end{array}$$

Step 3 - Matrix weighting.

Equation (5)

$$f_{ij} = r_{ij} \cdot \omega_j$$

$$w = \left[\frac{1}{7}, \frac{1}{7}, \frac{1}{7}, \frac{1}{7}, \frac{1}{7}, \frac{1}{7}, \frac{1}{7}\right]$$

$$Stand.\,decision\ matrix\ (R) = \begin{array}{c} \begin{array}{ccccccc} \textit{Trunk} & \textit{Max. Sp} & \textit{Power} & \textit{L/100} & \textit{Cost} & \textit{Seats} & \textit{Fuel} \end{array} \\ \begin{bmatrix} 0.4929 & 0.4532 & 0.7145 & -0.1102 & -0.6211 & 0.5916 & -0.2697 \\ 0.5027 & 0.4532 & 0.2568 & -0.5288 & -0.2945 & 0.4226 & -0.6742 \\ 0.4633 & 0.4029 & 0.5137 & -0.0000 & -0.5220 & 0.4226 & -0.1348 \\ 0.3894 & 0.4935 & 0.2802 & -0.6060 & -0.3577 & 0.3381 & -0.5394 \\ 0.3716 & 0.4381 & 0.2849 & -0.5839 & -0.3565 & 0.4226 & -0.4045 \end{bmatrix} \end{array} \begin{array}{l} \\ \textit{Car 1} \\ \textit{Car 2} \\ \textit{Car 3} \\ \textit{Car 4} \\ \textit{Car 5} \end{array}$$

$$Weighted\ normalized\ matrix\ (F) = \begin{array}{c} \begin{array}{ccccccc} \textit{Trunk} & \textit{Max. Sp} & \textit{Power} & \textit{L/100} & \textit{Cost} & \textit{Seats} & \textit{Fuel} \end{array} \\ \begin{bmatrix} 0.0704 & 0.0647 & 0.1021 & -0.0157 & -0.0877 & 0.0845 & -0.0385 \\ 0.0718 & 0.0647 & 0.0367 & -0.0755 & -0.0421 & 0.0604 & -0.0963 \\ 0.0662 & 0.0576 & 0.0734 & -0.0000 & -0.0746 & 0.0604 & -0.0193 \\ 0.0556 & 0.0705 & 0.0400 & -0.0866 & -0.0511 & 0.0483 & -0.0771 \\ 0.0531 & 0.0612 & 0.0407 & -0.0834 & -0.0509 & 0.0604 & -0.0578 \end{bmatrix} \end{array} \begin{array}{l} \\ \textit{Car 1} \\ \textit{Car 2} \\ \textit{Car 3} \\ \textit{Car 4} \\ \textit{Car 5} \end{array}$$

Step 4 - Determination of positive and negative ideal solutions

- With the weighted normalized matrix (*F*) and *equation (6)* the **positive** and **negative ideal solutions** can be selected.
- **Positive ideal solution (PIS)** → highest value of each criterion.
- **Negative ideal solution (NIS)** → lowest value of each criterion.

Equation 6

$$\begin{cases} f_j^+ = \boxed{max}\{f_{1j}, f_{2j}, f_{3j}, \dots f_{mj}\} \\ f_j^- = \boxed{min}\{f_{1j}, f_{2j}, f_{3j}, \dots f_{mj}\} \end{cases} \quad (j = 1, 2, 3, \dots n)$$

$$Weighted\ normalized\ matrix\ (F) = \begin{array}{c} \begin{array}{ccccccc} Trunk & Max.\ Sp & Power & L/100 & Cost & Seats & Fuel \end{array} \\ \begin{bmatrix} 0.0704 & 0.0647 & 0.1021 & -0.0157 & -0.0877 & 0.0845 & -0.0385 \\ 0.0718 & 0.0647 & 0.0367 & -0.0755 & -0.0421 & 0.0604 & -0.0963 \\ 0.0662 & 0.0576 & 0.0734 & -0.0000 & -0.0746 & 0.0604 & -0.0193 \\ 0.0556 & 0.0705 & 0.0400 & -0.0866 & -0.0511 & 0.0483 & -0.0771 \\ 0.0531 & 0.0612 & 0.0407 & -0.0834 & -0.0509 & 0.0604 & -0.0578 \end{bmatrix} \end{array} \begin{array}{c} Car\ 1 \\ Car\ 2 \\ Car\ 3 \\ Car\ 4 \\ Car\ 5 \end{array}$$

Step 4 - Determination of positive and negative ideal solutions

- ***PIS*** and ***NIS*** are defined with *equations (7)* and *(8)*

$$Weighted\ normalized\ matrix\ (F) = \begin{array}{c} \begin{array}{ccccccc} \textit{Trunk} & \textit{Max. Sp} & \textit{Power} & \textit{L/100} & \textit{Cost} & \textit{Seats} & \textit{Fuel} \end{array} \\ \begin{bmatrix} 0.0704 & 0.0647 & 0.1021 & -0.0157 & -0.0877 & 0.0845 & -0.0385 \\ 0.0718 & 0.0647 & 0.0367 & -0.0755 & -0.0421 & 0.0604 & -0.0963 \\ 0.0662 & 0.0576 & 0.0734 & -0.0000 & -0.0746 & 0.0604 & -0.0193 \\ 0.0556 & 0.0705 & 0.0400 & -0.0866 & -0.0511 & 0.0483 & -0.0771 \\ 0.0531 & 0.0612 & 0.0407 & -0.0834 & -0.0509 & 0.0604 & -0.0578 \end{bmatrix} \end{array} \begin{array}{l} \textit{Car 1} \\ \textit{Car 2} \\ \textit{Car 3} \\ \textit{Car 4} \\ \textit{Car 5} \end{array}$$

Equation (7) and (8)

$$PIS = \{f_1^+, f_2^+, f_3^+, \ldots f_7^+\}$$
$$NIS = \{f_1^-, f_2^-, f_3^-, \ldots f_7^-\}$$

Trunk	Max. Sp	Power	L/100	Cost	Seats	Fuel

$$PIS = \{0.0718;\ 0.0705;\ 0.1021; -0.0000; -0.0421; 0.0845; -0.0193\}$$

$$NIS = \{0.0531; 0.0576;\ 0.0367; -0.0866; -0.0887; 0.0483; -0.0963\}$$

Step 5 - Distance calculation to the positive- and negative-ideal solutions.

$$Weighted\ normalized\ matrix\ (F) = \begin{array}{c} \begin{array}{ccccccc} \textit{Trunk} & \textit{Max. Sp} & \textit{Power} & \textit{L/100} & \textit{Cost} & \textit{Seats} & \textit{Fuel} \end{array} \\ \begin{bmatrix} 0.0704 & 0.0647 & 0.1021 & -0.0157 & -0.0877 & 0.0845 & -0.0385 \\ 0.0718 & 0.0647 & 0.0367 & -0.0755 & -0.0421 & 0.0604 & -0.0963 \\ 0.0662 & 0.0576 & 0.0734 & -0.0000 & -0.0746 & 0.0604 & -0.0193 \\ 0.0556 & 0.0705 & 0.0400 & -0.0866 & -0.0511 & 0.0483 & -0.0771 \\ 0.0531 & 0.0612 & 0.0407 & -0.0834 & -0.0509 & 0.0604 & -0.0578 \end{bmatrix} \end{array} \begin{array}{l} \\ \textit{Car 1} \\ \textit{Car 2} \\ \textit{Car 3} \\ \textit{Car 4} \\ \textit{Car 5} \end{array}$$

Equation (9)

$$\begin{cases} d_i^+ = \sqrt{\sum_{j=1}^{n} \left(f_{ij} - f_j^+\right)^2} \\ d_i^- = \sqrt{\sum_{j=1}^{n} \left(f_{ij} - f_j^-\right)^2} \end{cases}$$

$$\begin{array}{l} \begin{array}{ccccccc} \textit{Trunk} & \textit{Max. Sp} & \textit{Power} & \textit{L/100} & \textit{Cost} & \textit{Seats} & \textit{Fuel} \end{array} \\ PIS = \{0.0718;\ 0.0705;\ 0.1021; -0.0000; -0.0421; 0.0845; -0.0193\} \\ NIS = \{0.0531; 0.0576;\ 0.0367; -0.0866; -0.0887; 0.0483; -0.0963\} \end{array}$$

$$d_i^+ = \begin{bmatrix} 0.0532 \\ 0.1286 \\ 0.0516 \\ 0.1278 \\ 0.1154 \end{bmatrix} \begin{array}{l} \textit{Car 1} \\ \textit{Car 2} \\ \textit{Car 3} \\ \textit{Car 4} \\ \textit{Car 5} \end{array} \qquad d_i^- = \begin{bmatrix} 0.1196 \\ 0.0534 \\ 0.1237 \\ 0.0444 \\ 0.0557 \end{bmatrix} \begin{array}{l} \textit{Car 1} \\ \textit{Car 2} \\ \textit{Car 3} \\ \textit{Car 4} \\ \textit{Car 5} \end{array}$$

Step 6 - Distance calculation to the positive- and negative-ideal solutions.

- CC_i (equation 10) shows how close an alternative i is to the positive-ideal solution and how far it is from the negative-ideal solution.

Equation (10)

$$Closeness\ coefficient\ (CC_i) = {d_i^-}/{(d_i^+ + d_i^-)}$$

$$d_i^+ = \begin{bmatrix} 0.0532 \\ 0.1286 \\ 0.0516 \\ 0.1278 \\ 0.1154 \end{bmatrix} \begin{matrix} \text{Car 1} \\ \text{Car 2} \\ \text{Car 3} \\ \text{Car 4} \\ \text{Car 5} \end{matrix} \qquad d_i^- = \begin{bmatrix} 0.1196 \\ 0.0534 \\ 0.1237 \\ 0.0444 \\ 0.0557 \end{bmatrix} \begin{matrix} \text{Car 1} \\ \text{Car 2} \\ \text{Car 3} \\ \text{Car 4} \\ \text{Car 5} \end{matrix}$$

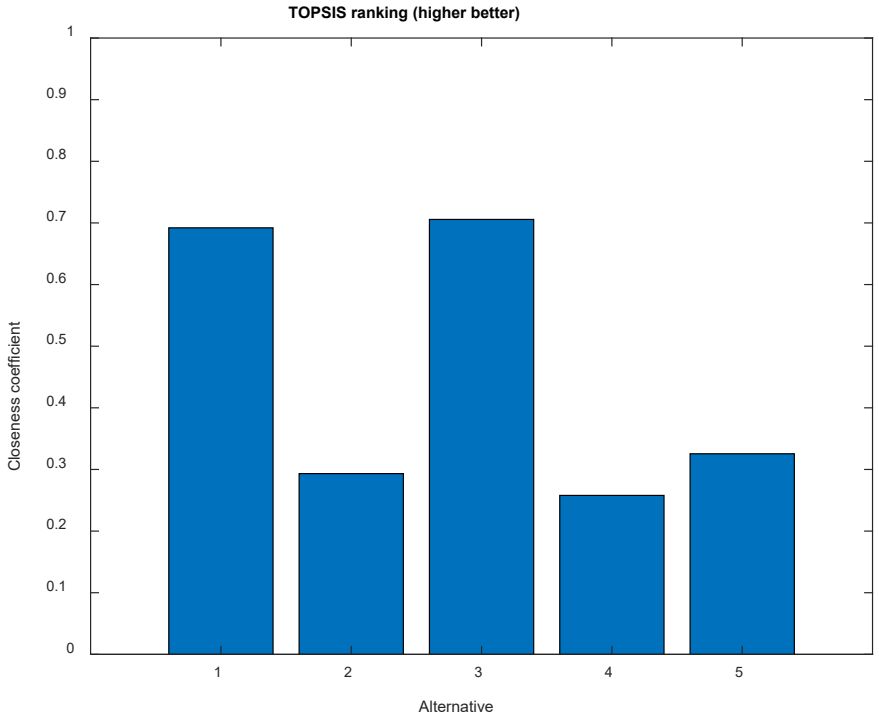
$$CC = \begin{bmatrix} 0.6920 \\ 0.2932 \\ 0.7057 \\ 0.2579 \\ 0.3255 \end{bmatrix} \begin{matrix} \text{Car 1} \\ \text{Car 2} \\ \text{Car 3} \\ \text{Car 4} \\ \text{Car 5} \end{matrix}$$

Sort CC values

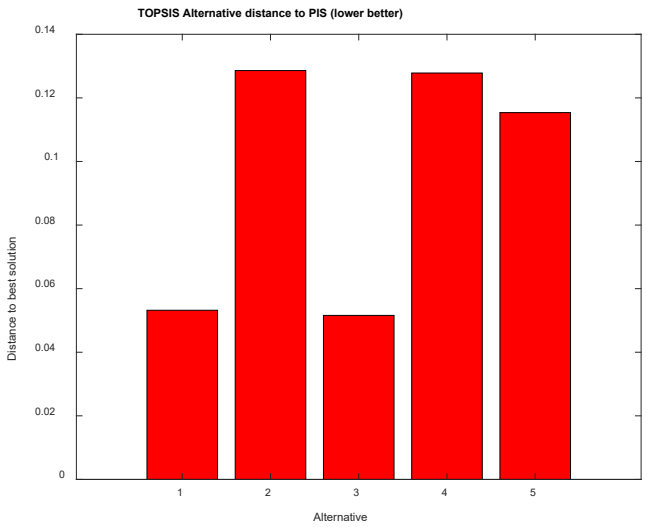
$$Ranking = \begin{bmatrix} Car\ 3 \\ Car\ 1 \\ Car\ 5 \\ Car\ 2 \\ Car\ 4 \end{bmatrix}$$

TOPSIS results:

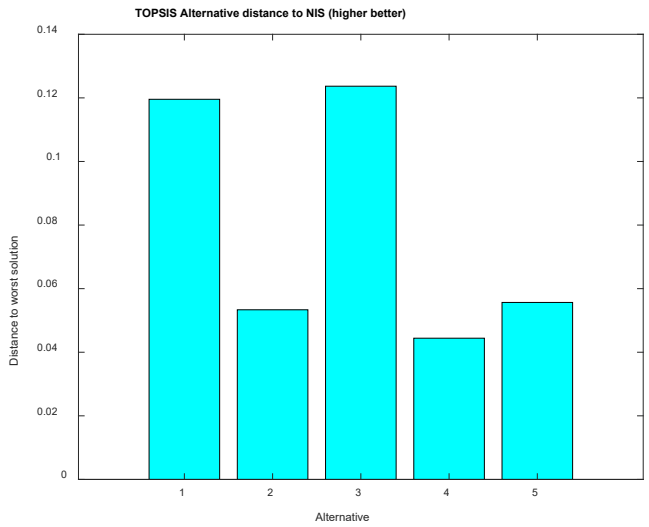
$$Ranking = \begin{bmatrix} Car\ 3 \\ Car\ 1 \\ Car\ 5 \\ Car\ 2 \\ Car\ 4 \end{bmatrix}$$



CC_i



d_i^+



d_i^-

Conclusions TOPSIS method:

- TOPSIS method is a valuable tool for selecting alternatives in different engineering fields. In this case study, TOPSIS shows that the best alternative is Car 3, but Car 1 is very close.

$$Ranking = \begin{bmatrix} Car\ 3 \\ Car\ 1 \\ Car\ 5 \\ Car\ 2 \\ Car\ 4 \end{bmatrix}$$

- It is important to highlight that Car 3 wins against the others with this combination of weights (equal weights $\omega = {}^{1}/_{7}$), and with another combination of weights this ranking could change.
- By systematically exploring the implications of each weight change on the final ranking, we gain valuable information to identify the optimal alternative.
- To understand the importance of weight selection in the final ranking of TOPSIS, we propose two different sensitivity analyses to study how the ranking is affected by changes in the weight values.

4.SENSITIVITY ANALYSIS

- Two sensitivity analyses methods were studied for a comprehensive assessment of the impact of the weights on the final ranking of the alternatives.

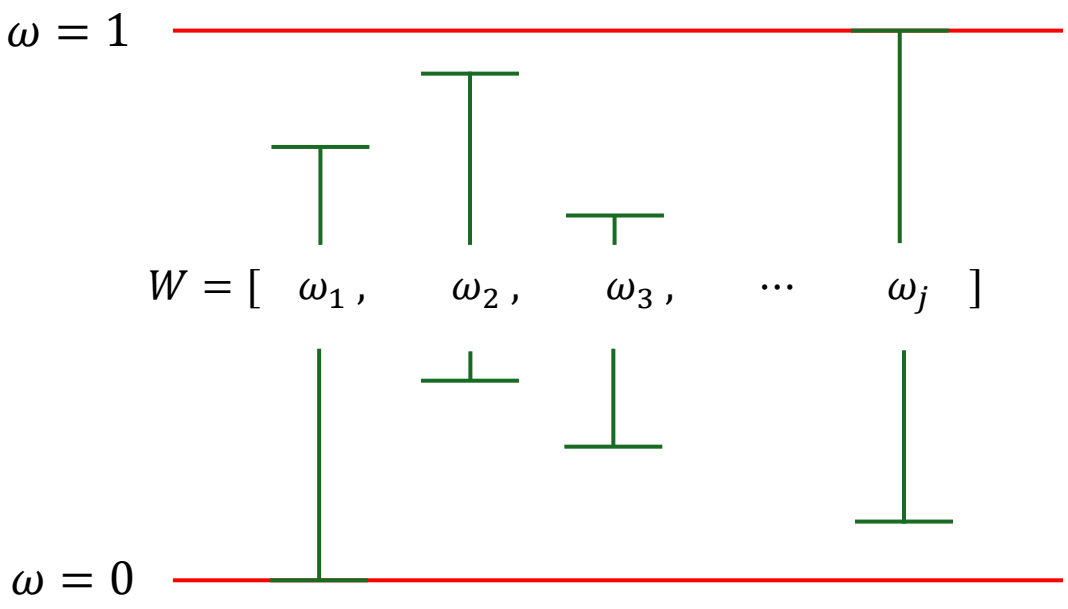
Weight Sensitivity Analysis (WSA)

How much a weight can be increased or decreased (adjusting the others proportionally) without changing which alternative is the top-ranked

Full Range Weight Sensitivity Analysis (FRWSA)

Tests all possible combinations of weights and analyses the prevalence of each alternative achieving the top rank

Weight Sensitivity Analysis (WSA)



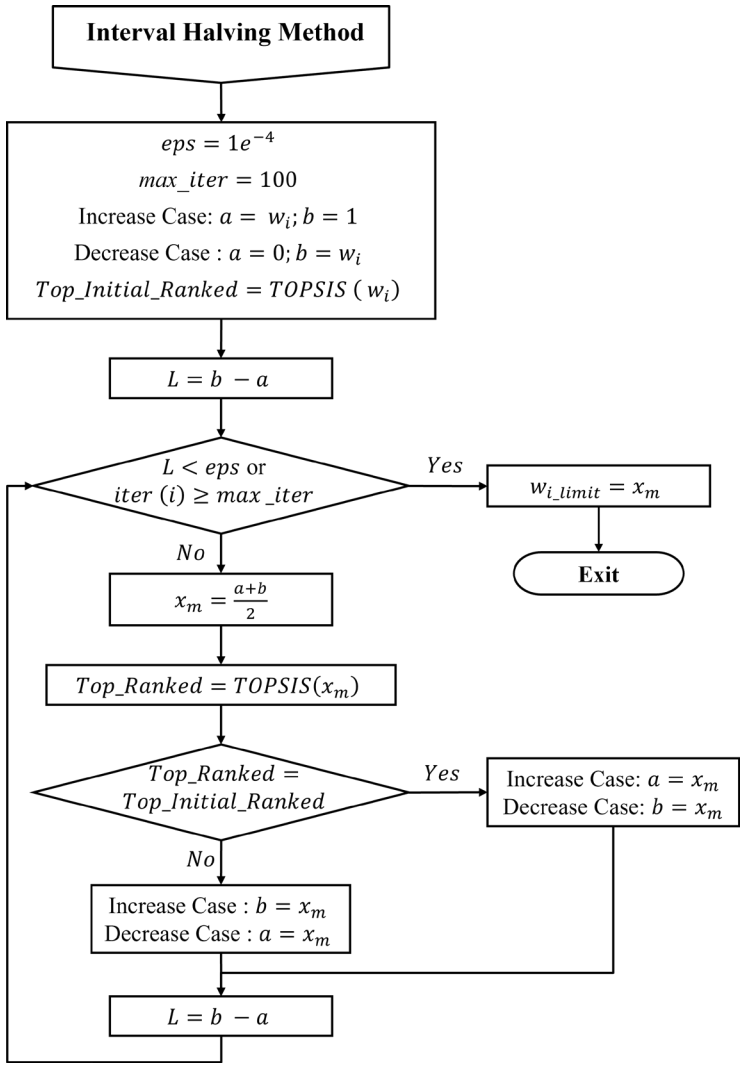
!

Without affecting the top-ranked alternative

Weight Sensitivity Analysis (WSA)

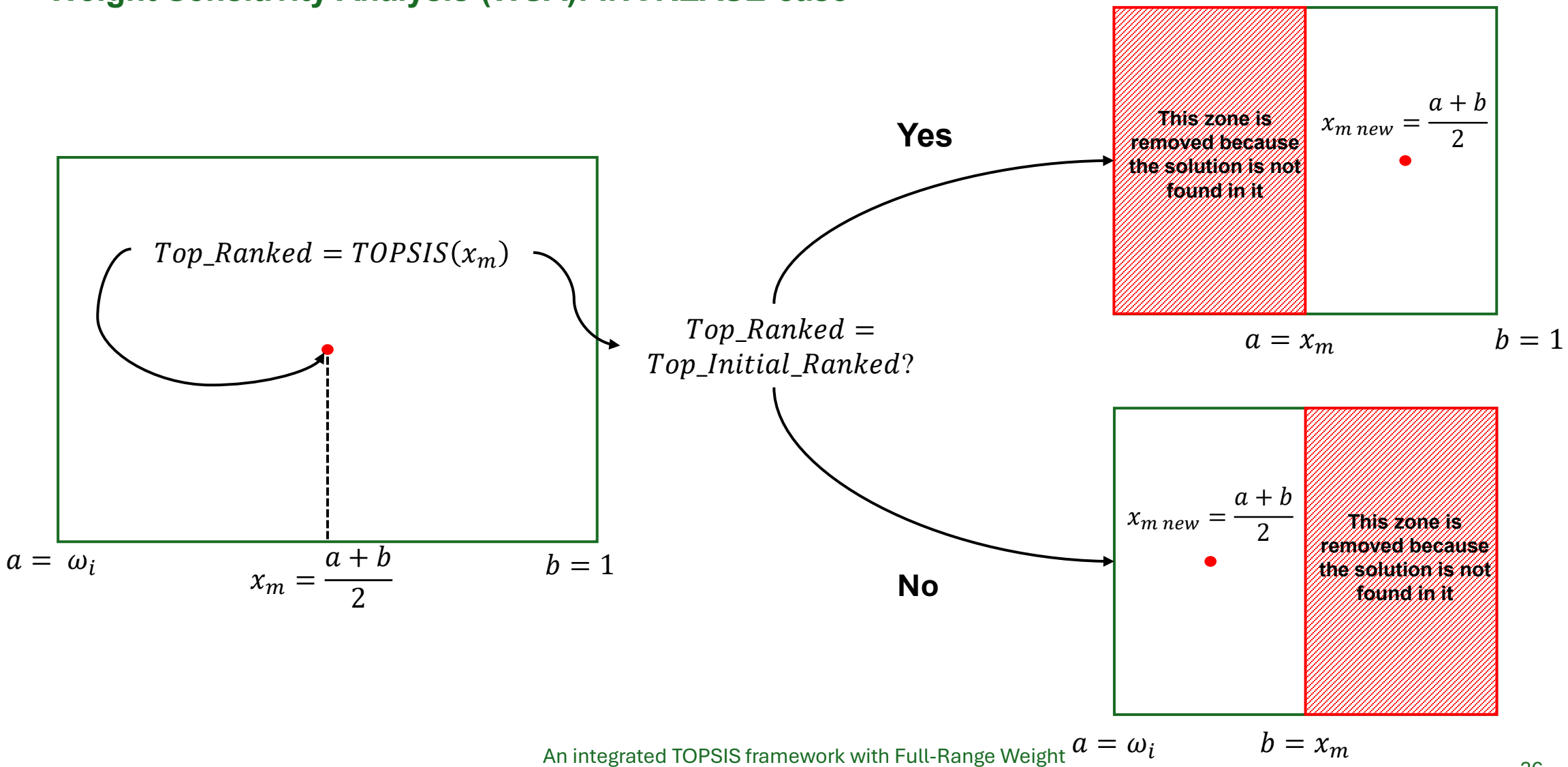
- Iterative method used → **Interval halving method**

- Classical iterative method for optimization that helps to reduce the range where the optimum is located.
- The iterative process is repeated until the desired level of accuracy is reached.
- The weight of each single criterion is either increased or decreased, while the weights of the remaining criteria are proportionally rescaled.
- The same procedure is sequentially applied to all the criteria.
- Reduces the number of iterations and computation time.

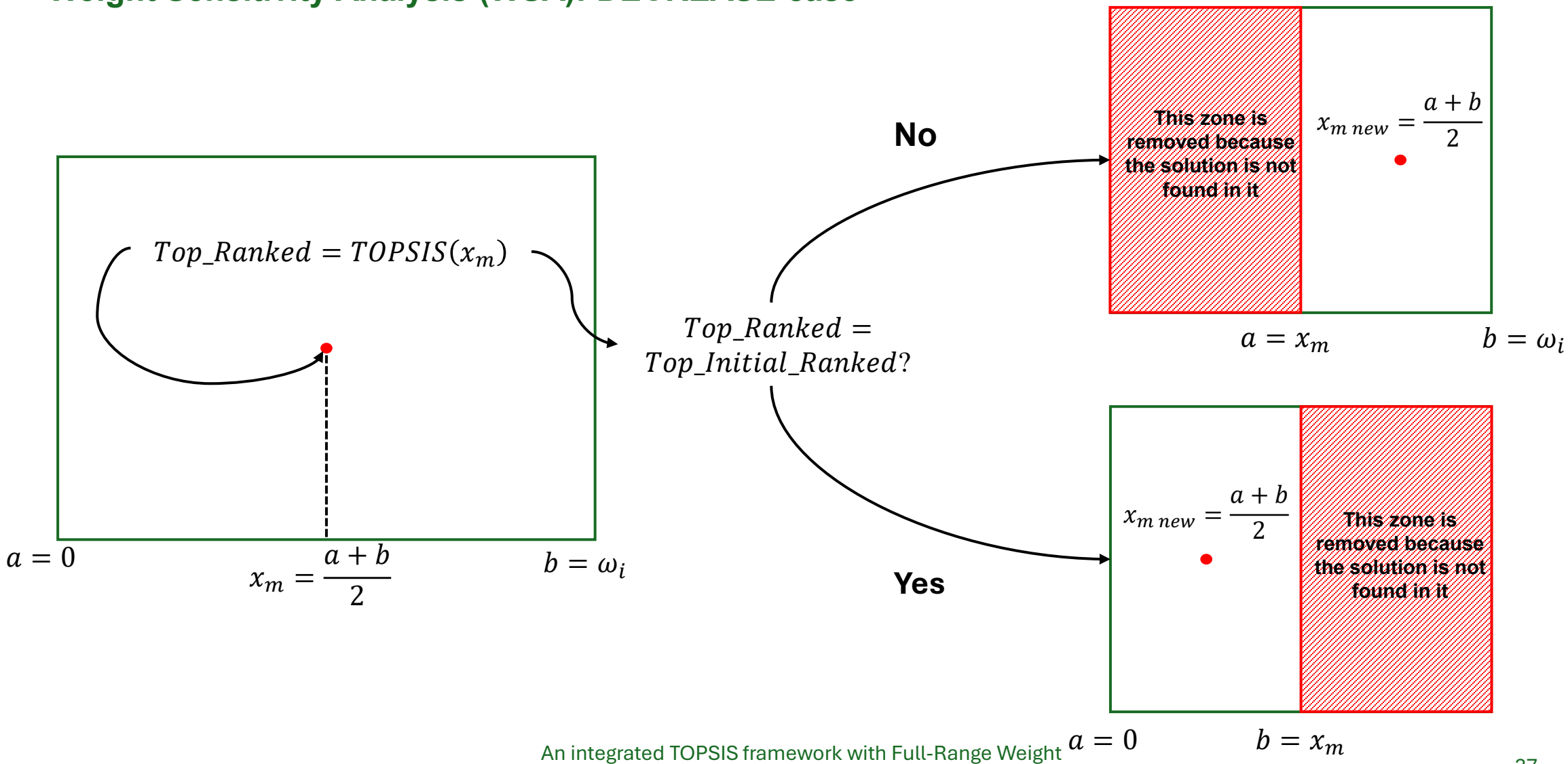


Flow chart of the interval halving method required in the WSA method.

Weight Sensitivity Analysis (WSA): INCREASE case



Weight Sensitivity Analysis (WSA): DECREASE case



User manual. Interval halving method

- This section explains how each step of the interval halving method has been programmed (*WSA_TOPSIS*), and the steps that the user must follow to use this method in a case study (*main_TOPSIS*).

main_TOPSIS

Users must complete **4 steps:**

1. Define the matrix C
2. Select the value of each of the weights, vector W
3. Define the vector MaxMin
4. Call the function *WSA_TOPSIS*

WSA_TOPSIS

MATLAB file containing the programmed WSA and interval halving method

User manual (main_TOPSIS):

- The first 3 steps to be followed by the user are the same as for the TOPSIS method:

1. ***Define decision matrix (C)***
2. ***Select the value of each of the weights, vector W***
3. ***Define the vector MaxMin:***

```
%% MAIN TOPSIS + WSA + FRWSA
%% CONSTRUCTION OF THE INITIAL DECISION MATRIX (C)
% DECISION MATRIX (C)

% Comparison of five cars and seven criteria
 %Trunk(L) MaxSp(km/h) Power(hp) L/100km Cost  N.Seats  Combust.
C = [500      180       306      1     51750    7         2;   % CAR 1
     510      180       110     4.8    24540    5         5;   % CAR 2
     470      160       220      0     43488    5         1;   % CAR 3
     395      196       120     5.5    29800    4         4;   % CAR 4
     377      170       122     5.3    29700    5         3];  % CAR 5

%% Weight vector (W) - Relative importance of each criterion
[N_Alt,N_Crit] = size(C);
W = ones(1,N_Crit)/N_Crit; %Same weight for all the criteria
% wi = 1 / 7; %
% W = [wi wi wi wi wi wi wi]; Other way to define the W vector

%% MaxMin vector (MM) - Direction of impact for each criterion (1 to maximize, -1 to minimize)
% In other words, here it is important if it is more favorable that the studied value is larger (1) or smaller (-1).
MM = [1 1 1 -1 -1 1 -1];
```

User manual (main_TOPSIS):

4. Call the function WSA_TOPSIS

```
%% WSA
WSA_TOPSIS(C,W,MM)
```

- WSA_TOPSIS calls twice half_TOPSIS, one for the INCREASE case, and one for the DECREASE case.
- half_TOPSIS receives as input:
  - decision matrix (*C*), vector of weights (*W*), vector MaxMin (*MM)*
  - INCREASE (1) or DECREASE (0) case
  - Tol: Tolerance for finding the weight limits
  - MaxIt: Maximum number of Iterations
- And sends as output:
  - The minimum or maximum of each weight that does not change the initial ranking

```
function WSA_TOPSIS(C,W,MM)

Tol = 1e-5;
MaxIt = 100;

[resUP]=half_TOPSIS(C,W,MM,1,Tol,MaxIt,0);
[resDOWN]=half_TOPSIS(C,W,MM,0,Tol,MaxIt,0);

% RESULTS
disp("Sensitivity Analysis of Weights (W)");
disp(['Max:     ' num2str(resUP, '%.3f ')]);
disp(['Default:' num2str(W, '%.3f ')]);
disp(['Min:     ' num2str(resDOWN, '%.3f ')]);
figure(4);
[~, N_Crit] = size(C);
x=1:N_Crit;
plot(x,W, 'x',x, resUP, 'vr', x, resDOWN, '^b');
hold on
for j=1:N_Crit
    plot([j, j], [resUP(j) resDOWN(j)],'-k')
end
axis([0 N_Crit+1 0 1]);
legend('Default weight', 'Max weight', 'Min weight','Possible values'
xlabel('Criterion')
ylabel('Weight')
title('Weight sensitivity analysis of the best alternative')
```

User manual (half_TOPSIS):

- Steps to implement the MATLAB code interval halving method:
  1. Initialise (*zeros*) the size of the matrix that will contain the results, equal to the number of weights in the study.
  2. **Initial TOPSIS ranking** calculation using the weights given as input.
  3. Loop (*for*) to calculate the method from the first weight value to the last.
  - Iteration counter (start = 0). Every change of weight of this loop this counter is reset to 0.
  - Calculation of interval limits (a,b).

```
function res = half_TOPSIS(C, W, I, lim, eps, it, stats)
if nargin == 7
    if    (stats ~= 1)
        stats = 0;
    end
else
    stats = 0;
end
res = zeros(size(W));

[rankingINI,~,~,~] = fun_TOPSIS(C,W,I);  % call to the TOPSIS function, to see the initial ranking.
for i = 1:length(W)
    iter = 0;

    % Determine interval limits
    a = min(W(i),lim);
    b = max(W(i),lim);
    L = b - a;

    Wm = W;
```

INCREASE CASE (lim = 1)

$$a = \min(\omega_i, 1) = \omega_i$$
$$b = \max(\omega_i, 1) = 1$$

DECREASE CASE (lim = 0)

$$a = \min(\omega_i, 0) = 0$$
$$b = \max(\omega_i, 0) = \omega_i$$

User manual (half_TOPSIS):

3. Loop to calculate the method for each weight.

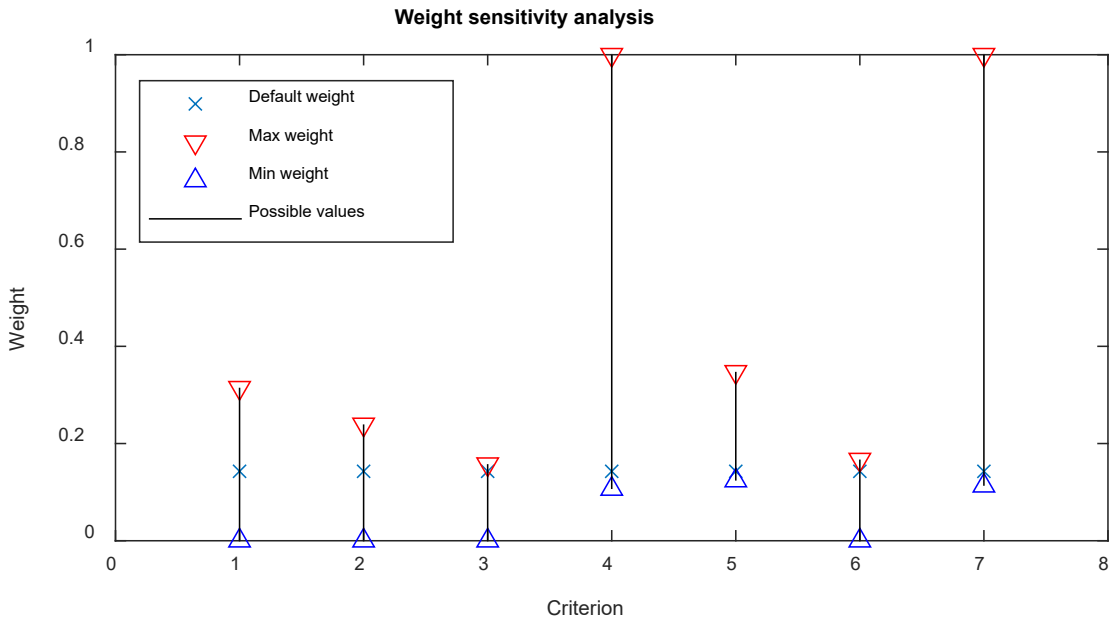
- The MATLAB *while* instruction will start the iterative process (will end when the tolerance is reached or when the maximum number of iterations has been exceeded).
- Midpoint calculation (*xm*).
- Rescale to preserve a cumulative weight sum of one.
- TOPSIS ranking calculation with the new rescaled weights.
- Interval halving method.
- Add one to iteration counter
- Information about each iteration (*iter*: number of iteration, *a* and *b* values, rankingm: top-ranked alternative in each iteration.
- Result *(res(i))* → how much the weight *i* can increase or decrease when the desired tolerance has been reached or the maximum number of iterations has been reached.

```
while (L > eps && iter < it)
    xm = (a + b) / 2;
    Wm(i) = xm;
    Rem = 1 - xm;
    Rest = sum(Wm) - xm;
    Wm = Wm * Rem / Rest;
    Wm(i) = xm;

    [rankingm,~,~,~] = fun_TOPSIS(C,Wm,I);
    if rankingINI(1) == rankingm(1)
        if lim==1 %Case increase
            a = xm;
        else %Case decrease
            b = xm;
        end
    else
        if lim==1 %Case increase
            b = xm;
        else %Case decrease
            a = xm;
        end
    end

    L = b - a;
    iter = iter + 1;
    if (stats)
        fprintf('%d\t%4.3f\t%4.3f\t%d\n', iter, a, b, rankingm(1));
    end
end
if (stats)
    fprintf('\n');
end
res(i) = xm;
end
```

Weight Sensitivity Analysis (WSA) – Car results

Sensitivity Analysis of Weights (W)

	w1	w2	w3	w4	w5	w6	w7
Max:	0.315	0.239	0.158	1.000	0.347	0.167	1.000
Default:	0.143	0.143	0.143	0.143	0.143	0.143	0.143
Min:	0.000	0.000	0.000	0.106	0.124	0.000	0.113



Trunk Max.Sp Power L/100 Cost Seats Fuel

- The weights of criteria 4 and 7 can be modified in a wide range up to 1, as for both criteria Car 3 is the best alternative.
- Criteria 3 and 6 have a small range of possible weights. Car 3 is not the best for these criteria, so if these weights are increased, Car 3 will be outcompeted by other alternatives. Only by increasing them from 0.143 to 0.170 is Car 3 no longer the best alternative.

Ranges of weights without affecting the top-ranked alternative (Car 3)

Full Range Weight Sensitivity Analysis (FRWSA):

- FRWSA explores all feasible combinations of weighting vectors where the weights sum to one. TOPSIS is evaluated for each combination of weights ranging from 0 to 1 in predefined steps and with $\sum_{i=1}^{n.criteria} w_i = 1$.
- The size of this matrix (*equation 11*) depends on the interval chosen and on the number of alternatives.

Matrix (*WMAT*) dimensions

Rows = depends on the interval (0-1)
Columns = # Criteria (*j*)

$$Weight\ Matrix(WMAT) = \begin{array}{c} \begin{array}{ccccccc} \omega_1 & \omega_2 & \omega_3 & \omega_4 & & \omega_{j-1} & \omega_j \end{array} \\ \left[\begin{array}{ccccccc|c} 1 & 0 & 0 & 0 & \dots & 0 & 0 & 1 \\ 0.95 & 0.05 & 0 & 0 & \dots & 0 & 0 & 1 \\ \vdots & \vdots & \vdots & \vdots & \ddots & 0 & \vdots & \vdots \\ 0 & 0 & 0 & 1 & \dots & 0 & 0 & 1 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & 0.05 & 0.95 & 1 \\ 0 & 0 & 0 & 0 & \dots & 0 & 1 & 1 \end{array}\right] \end{array} \quad (11)$$

The sum of the weights for each case adds up to 1

Full Range Weight Sensitivity Analysis (FRWSA):

- **Matrix (WMAT) dimensions**. As an example, this table shows, for an interval of 0.05, the number of possible combinations for different number of criteria studied (2 - 10), i.e. the number of rows in WMAT.

Possible combinations for different numbers of criteria studied.

Number of criteria	MATLAB function name	Number of combinations
2	Wmatrix_2	21
3	Wmatrix_3	231
4	Wmatrix_4	1,171
5	Wmatrix_5	10,626
6	Wmatrix_6	53,130
7	Wmatrix_7	230,230
8	Wmatrix_8	888,030
9	Wmatrix_9	3,108,105
10	Wmatrix_10	10,015,005

Full Range Weight Sensitivity Analysis (FRWSA).

- For each of the possible combinations of weights, the TOPSIS method is applied.

$$Weight\ Matrix(WMAT) = \begin{array}{c} \begin{array}{ccccccc} \omega_1 & \omega_2 & \omega_3 & \omega_4 & & \omega_{j-1} & \omega_j \end{array} \\ \begin{bmatrix} 1 & 0 & 0 & 0 & \dots & 0 & 0 \\ 0.95 & 0.05 & 0 & 0 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & 0 & \vdots \\ 0 & 0 & 0 & 1 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & 0.05 & 0.95 \\ 0 & 0 & 0 & 0 & \dots & 0 & 1 \end{bmatrix} \end{array} \begin{array}{c} 1 \\ 1 \\ \vdots \\ 1 \\ \vdots \\ 1 \\ 1 \end{array} \quad (11)$$

Matrix (*RankMAT*) dimensions

Rows = # Alternatives
Columns = # Rows (WMAT)

$$Ranking\ Matrix\ (RankMAT) = \begin{bmatrix} TOP\ 1\ (WMAT(1,:)) & TOP\ 1\ (WMAT(2,:)) & \cdots & TOP\ 1\ (WMAT(n.Comb,:)) \\ TOP\ 2\ (WMAT(1,:)) & TOP\ 2\ (WMAT(2,:)) & \cdots & TOP\ 2\ (WMAT(n.Comb,:)) \\ TOP\ 3\ (WMAT(1,:)) & TOP\ 3\ (WMAT(2,:)) & \cdots & TOP\ 3\ (WMAT(n.Comb,:)) \\ \vdots & \vdots & \vdots & \vdots \\ TOP\ n.Alt\ (WMAT(1,:)) & TOP\ n.Alt\ (WMAT(2,:)) & \cdots & TOP\ n.Alt\ (WMAT(n.Comb,:)) \end{bmatrix} \quad (12)$$

Full Range Weight Sensitivity Analysis (FRWSA).

- With all rankings for all possible scenarios, the results are presented through different graphs to better communicate these insights.

$$Ranking\ Matrix\ (RankMAT) = \begin{bmatrix} TOP\ 1\ (WMAT(1,:)) & TOP\ 1\ (WMAT(2,:)) & \cdots & TOP\ 1\ (WMAT(n.Comb,:)) \\ TOP\ 2\ (WMAT(1,:)) & TOP\ 2\ (WMAT(2,:)) & \cdots & TOP\ 2\ (WMAT(n.Comb,:)) \\ TOP\ 3\ (WMAT(1,:)) & TOP\ 3\ (WMAT(2,:)) & \cdots & TOP\ 3\ (WMAT(n.Comb,:)) \\ \vdots & \vdots & \vdots & \vdots \\ TOP\ n.Alt\ (WMAT(1,:)) & TOP\ n.Alt\ (WMAT(2,:)) & \cdots & TOP\ n.Alt\ (WMAT(n.Comb,:)) \end{bmatrix} \quad (15)$$

Pie plot

Frequency Distribution plot of the ranking position

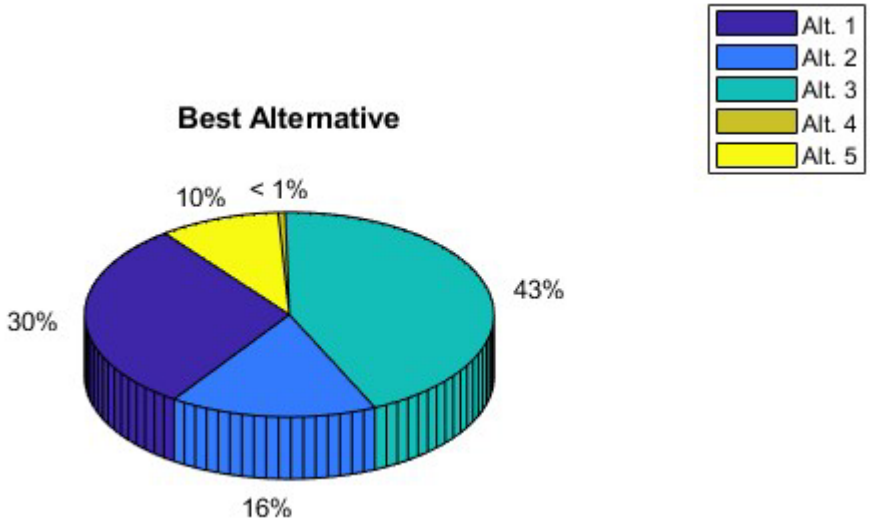
FRWSA plot

Full Range Weight Sensitivity Analysis (FRWSA).

- ***Pie plot***: It represents the fraction of scenarios that each alternative is the top-ranked. For this purpose, the frequency of each alternative appearing in the first row of *RankMAT* is calculated.

$$Ranking\ Matrix\ (RankMAT) = \begin{bmatrix} TOP\ 1\ (WMAT(1,:)) & TOP\ 1\ (WMAT(2,:)) & \cdots & TOP\ 1\ (WMAT(n.Comb,:)) \\ TOP\ 2\ (WMAT(1,:)) & TOP\ 2\ (WMAT(2,:)) & \cdots & TOP\ 2\ (WMAT(n.Comb,:)) \\ TOP\ 3\ (WMAT(1,:)) & TOP\ 3\ (WMAT(2,:)) & \cdots & TOP\ 3\ (WMAT(n.Comb,:)) \\ \vdots & \vdots & \vdots & \vdots \\ TOP\ n.Alt\ (WMAT(1,:)) & TOP\ n.Alt\ (WMAT(2,:)) & \cdots & TOP\ n.Alt\ (WMAT(n.Comb,:)) \end{bmatrix} \quad (15)$$

Calculation of frequency for each alternative



Full Range Weight Sensitivity Analysis (FRWSA).

- ***Frequency distribution plot of the ranking position***: This heat map shows the frequency of each alternative in each ranking position.

$$Ranking\ Matrix\ (RankMAT) = \begin{bmatrix} TOP\ 1\ (WMAT(1,:)) & TOP\ 1\ (WMAT(2,:)) & \cdots & TOP\ 1\ (WMAT(n.Comb,:)) \\ TOP\ 2\ (WMAT(1,:)) & TOP\ 2\ (WMAT(2,:)) & \cdots & TOP\ 2\ (WMAT(n.Comb,:)) \\ TOP\ 3\ (WMAT(1,:)) & TOP\ 3\ (WMAT(2,:)) & \cdots & TOP\ 3\ (WMAT(n.Comb,:)) \\ \vdots & \vdots & \vdots & \vdots \\ TOP\ n.Alt\ (WMAT(1,:)) & TOP\ n.Alt\ (WMAT(2,:)) & \cdots & TOP\ n.Alt\ (WMAT(n.Comb,:)) \end{bmatrix} \quad (15)$$

Calculation of the frequency of each alternative for each rank

	Rank 1	Rank 2	Rank 3	Rank n
Alt. 1	% Alt 1→TOP 1	% Alt 1→TOP 2	% Alt 1→TOP 3	% Alt 1→TOP n
Alt. 2	% Alt 2→TOP 1	% Alt 2→TOP 2	% Alt 2→TOP 3	% Alt 2→TOP n
Alt. 3	% Alt 3→TOP 1	% Alt 3→TOP 2	% Alt 3→TOP 3	% Alt 3→TOPn
Alt. n	% Alt n→TOP 1	% Alt n→TOP 2	% Alt n→TOP 3	% Alt n→TOP n

Full Range Weight Sensitivity Analysis (FRWSA).

- ***FRWSA plot***: Histogram graph plot for each alternative where the fraction of scenarios in which the alternative (e.g. *k)* is the best option for each criterion *j* and specific weighting value *i*.

$$Ranking\ Matrix\ (RankMAT) = \begin{bmatrix} TOP\ 1\ (WMAT(1,:)) & TOP\ 1\ (WMAT(2,:)) & \cdots & TOP\ 1\ (WMAT(n.Comb,:)) \\ TOP\ 2\ (WMAT(1,:)) & TOP\ 2\ (WMAT(2,:)) & \cdots & TOP\ 2\ (WMAT(n.Comb,:)) \\ TOP\ 3\ (WMAT(1,:)) & TOP\ 3\ (WMAT(2,:)) & \cdots & TOP\ 3\ (WMAT(n.Comb,:)) \\ \vdots & \vdots & \vdots & \vdots \\ TOP\ n.Alt\ (WMAT(1,:)) & TOP\ n.Alt\ (WMAT(2,:)) & \cdots & TOP\ n.Alt\ (WMAT(n.Comb,:)) \end{bmatrix} \quad (15)$$

- For each alternative, the *RankMAT* matrix is used to select the positions where that alternative *k* has been top-ranked.

$$WRank1Alt(k) = WMAT(columnRank1,:) \quad (16)$$

- $WRank1Alt(k)$, contains all combinations of weights where alternative k has been top-ranked.

Full Range Weight Sensitivity Analysis (FRWSA).

- The matrix *FRMAT* contains the fraction of all possible cases in which alternative *k* is top-ranked for each criterion *j* and specific weighting value *i* (equation 17).

(17)

$$FRMAT(i,j,1) = \begin{bmatrix} frc_{111} & frc_{121} & frc_{131} & \cdots & frc_{1j1} \\ frc_{211} & frc_{221} & frc_{231} & \cdots & frc_{2j1} \\ frc_{311} & frc_{321} & frc_{331} & \cdots & frc_{3j1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ frc_{i11} & frc_{i21} & frc_{i31} & \cdots & frc_{ij1} \end{bmatrix}$$

$$FRMAT(i,j,2) = \begin{bmatrix} frc_{112} & frc_{122} & frc_{132} & \cdots & frc_{1j2} \\ frc_{212} & frc_{222} & frc_{232} & \cdots & frc_{2j2} \\ frc_{312} & frc_{322} & frc_{332} & \cdots & frc_{3j2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ frc_{i12} & frc_{i22} & frc_{i32} & \cdots & frc_{ij2} \end{bmatrix}$$

$$FRMAT(i,j,3) = \begin{bmatrix} frc_{113} & frc_{123} & frc_{133} & \cdots & frc_{1j3} \\ frc_{213} & frc_{223} & frc_{233} & \cdots & frc_{2j3} \\ frc_{313} & frc_{323} & frc_{333} & \cdots & frc_{3j3} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ frc_{i13} & frc_{i23} & frc_{i33} & \cdots & frc_{ij3} \end{bmatrix}$$

$$FRMAT(i,j,k) = \begin{bmatrix} frc_{11k} & frc_{12k} & frc_{13k} & \cdots & frc_{1jk} \\ frc_{21k} & frc_{22k} & frc_{23k} & \cdots & frc_{2jk} \\ frc_{31k} & frc_{32k} & frc_{33k} & \cdots & frc_{3jk} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ frc_{i1k} & frc_{i2k} & frc_{i3k} & \cdots & frc_{ijk} \end{bmatrix}$$

k = k (Alt. k)

k = 3 (Alt. 3)

k = 2 (Alt. 2)

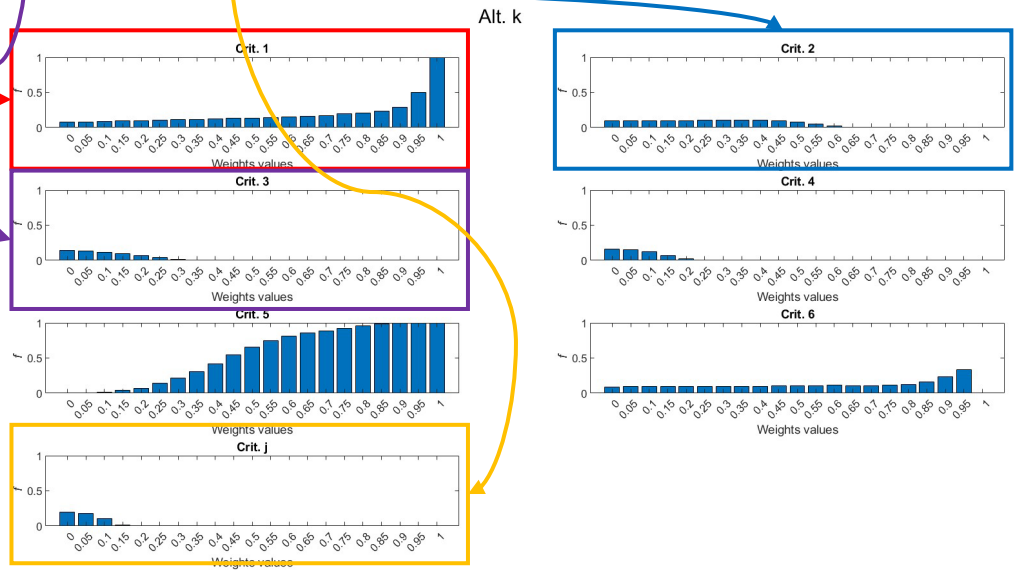
k = 1 (Alt. 1)

Full Range Weight Sensitivity Analysis (FRWSA).

- The fraction of scenarios in which alternative *k* is the best option for each criterion *j* and specific weighting value *i* is calculated.

$$FRMAT(i,j,k) = \begin{bmatrix} frc_{11k} & frc_{12k} & frc_{13k} & \cdots & frc_{1jk} \\ frc_{21k} & frc_{22k} & frc_{23k} & \cdots & frc_{2jk} \\ frc_{31k} & frc_{32k} & frc_{33k} & \cdots & frc_{3jk} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ frc_{i1k} & frc_{i2k} & frc_{i3k} & \cdots & frc_{ijk} \end{bmatrix} \quad (17)$$

The results are plotted in a histogram plot for each alternative k



User manual. Full Range Weight Sensitivity Analysis

- In this section we explain how each step of the **Full Range Weight Sensitivity Analysis (FRWSA)** has been programmed (*FRWSA_TOPSIS*), and the steps that the user must follow in order to implement this method in a case study (*main_TOPSIS*).

main_TOPSIS_cars	*Wmatrix_i*	*FRWSA_TOPSIS*
The user must perform **5 steps:** 1. Define the matrix C 2. Define the vector MaxMin 3. Name each alternative 4. Name each criterion 5. Call the function *FRWSA_TOPSIS.*	MATLAB functions providing the calculation of the matrix containing all combinations of weights for *i* criteria (*i*=2 to 10).	MATLAB file containing programmed FRWSA method.

User manual (main_TOPSIS_cars):

- The first 2 steps to be followed by the user are the same as for the TOPSIS method:

1. ***Define decision matrix (C)***
2. ***Define the vector MaxMin***

```
%% MAIN FRWSA TOPSIS
%% CONSTRUCTION OF THE INITIAL DECISION MATRIX (C)
% DECISION MATRIX (C)

% Comparison of five cars and seven criteria
 %Trunk(L) MaxSp(km/h) Power(hp) L/100km Cost  N.Seats  Combust.
C = [500      180       306      1     51750    5         2;   % CAR 1
     510      180       110     4.8    24540    5         5;   % CAR 2
     470      160       220      0     43488    5         1;   % CAR 3
     395      196       120     5.5    29800    5         4;   % CAR 4
     377      170       122     5.3    29700    5         3];  % CAR 5

%% MaxMin vector (MM) - Direction of impact for each criterion (1 to maximize, -1 to minimize)
% In other words, here it is important if it is more favorable that the studied value is larger (1) or smaller (-1).
MM = [1 1 1 -1 -1 1 -1];
```

User manual (main_TOPSIS_cars):

3. ***Name each alternative:***
   - List the names of each alternative (Alternative 1 → Alternative k), as a vector.

```
Alt_Names = {'Alt. 1','Alt. 2 ','Alt. 3','Alt. 4','Alt. k'};
```

4. ***Name each criterion:***
   - List the names of each criterion (Criterion 1 → Criterion k), as a vector.

```
Crit_Names = {'Crit. 1','Crit. 2','Crit. 3','Crit. 4', 'Crit. 5','Crit. 6','Crit. j'};
```

These labels will be useful for plotting the graphs and giving names to the axes of the alternatives and criteria

User manual (main_TOPSIS_cars):

5. Call the function FRWSA_TOPSIS:

- FRWSA_TOPSIS receives as input:
  - *C*: decision matrix
  - *MM*: vector MaxMin
  - *Npoints*: Number of points in the 0-1 interval of weights. For instance, with 21 points, the weight range will be from 0 to 1 in 0.05 steps.
  - *Alt_Names, Crit_Names*: Labels used in the figures

- There is no specific numerical output, but it provides the three graphs described above

```
%% FRWSA
Alt_Names = {'CAR 1','CAR 2 ','CAR 3','CAR 4','CAR 5'};
Crit_Names = {'Trunk','MaxSp','Power','L/100km', 'Cost','N.Seats','Combust.'};
Npoints=21;

FRWSA_TOPSIS(C,MM,Npoints,Alt_Names,Crit_Names)
```

User manual (Wmatrix_i):

- Functions *Wmatrix_i* were programmed for defining the weighting matrix with all possible combinations for n (2-10) criteria:
  - $n\omega$: Number of values a weight can have.
  - $max\omega$: Maximum value a weight can have (Typically $max\omega = 1$).
  - $Weights$: Possible values a weight can have.
  - idx: Iteration counter which will correspond to each combination of weights (and will represent the index of the corresponding row).

$$
\begin{aligned}
&for\ \omega_1 = 1{:}\,n\omega \\
&\quad for\ \omega_2 = 1{:}\,n\omega - \omega_1 + 1 \\
&\qquad \vdots \\
&\qquad for\ \omega_{n-1} = 1{:}\,n\omega - \omega_1 - \omega_2 - \cdots - \omega_{n-1} - (n-2) \\
&\qquad\quad \omega_n = max\omega - Weights(\omega_1) - Weights(\omega_2) - \cdots - Weights(\omega_{n-1}) \\
&\qquad\quad \omega matrix(idx) = [Weights(\omega_1)\ Weights(\omega_2) \ldots Weights(\omega_{n-1})\ \omega_n] \\
&\qquad\quad idx = idx + 1 \\
&\qquad end \\
&\quad end \\
&end
\end{aligned}
$$

User manual (Wmatrix_i):

To explain the code, an example is given for a matrix with 7 different criteria (Wmatrix_7):

1. Inputs:
   I. maxW: Maximum value that a weight can have
   II. nW: Number of levels of weight desired
2. Outputs:
   I. Wmatrix: Matrix containing all weight combinations
   II. rep: vector providing information about the number of times that each of the different values that a weight can have are repeated in a criterion
   III. numCombinations: number of combinations
3. Calculation of the number of possible combinations

   numCombinations = ***nchoosek***(nW+num-2,nW-1). Nchoosek: is used to calculate the number of possible combinations.
4. Initialise the matrix

```
function [Wmatrix,rep,numCombinations] = Wmatrix7(maxW, nW)

%% ALL POSSIBLE COMBINATIONS
num = 7;
numCombinations = nchoosek(nW+num-2,nW-1);
Wmatrix = zeros(numCombinations, 8);
Weights = 0:maxW/(nW-1):maxW;
idx = 1;
```

User manual (Wmatrix_i):

5. Calculation of the weight matrix
6. Calculation of the number of occurrences of a specific weight in the matrix

```
% CONSTRUCTION OF THE n-DIMENSIONAL MATRIX
for W1 = 1:nW
    for W2 = 1:nW-W1+1
        for W3 = 1:nW-W1-W2+2
            for W4 = 1:nW-W1-W2-W3+3
                for W5 = 1:nW-W1-W2-W3-W4+4
                    for W6 = 1:nW-W1-W2-W3-W4-W5+5
                        W7 = maxW - Weights(W1) - Weights(W2) - Weights(W3) - Weights(W4) - Weights(W5) - Weights(W6);
                        Wmatrix(idx, 1:7) = [Weights(W1) Weights(W2) Weights(W3) Weights(W4) Weights(W5) Weights(W6) W7];
                        Wmatrix(idx,8)=sum(Wmatrix(idx,:));
                        idx = idx + 1;
                    end
                end
            end
        end
    end
end

% CALCULATION OF HOW MANY TIMES A SPECIFIC WEIGHT APPEARS IN THE MATRIX
rep=zeros(nW,1);
for i=1:nW
    rep(i)=nchoosek(nW+num-2-i,num-2);
end
end
```

User manual (FRWSA_TOPSIS):

- Steps implementing the MATLAB code FRWSA TOPSIS:
  1. Calculate the increment of weight.
  2. Call the function *Wmatrix_i*. Depending on the number of criteria in the study this function will call the appropriate function. For example, if there are 5 criteria it will call the function Wmatrix_5. The result is the WMAT matrix, where each row of this matrix corresponds to each possible combination of weights.
  3. Call the TOPSIS function. A *For* loop is performed to calculate the ranking for each of the possible combinations of weights.

```
function FRWSA_TOPSIS(Matrix_C,MM,Npoints,Alt_Names,W_Names)
%% Number of alternatives and criteria
[N_Alt, N_Crit] = size(Matrix_C);

%% Possible weights
% Call the function to get the whole combination of weights
Incr = 1/(Npoints-1); % Increment of- weight

%% Selecting the Weight Matrix depending on the number of criteria
functionName = ['Wmatrix_', num2str(N_Crit)];
[Weights_Matrix, repW, numCombinations] = feval(functionName, 1, Npoints);
WMAT = Weights_Matrix(:,1:N_Crit);
disp(['FRWSA evaluating: ' num2str(numCombinations, '%1 ') ' weight combinat
%% TOPSIS Fun
[n,~]=size(WMAT);
RankMAT=zeros(N_Alt,n);
for j = 1:length(WMAT(:,1))
    W = WMAT(j,:);
    [ranking,~,~,~] = fun_TOPSIS(Matrix_C,W,MM);   % TOPSIS Function Call.
    RankMAT(:,j) = ranking;                        % According to equation
end
```

User manual (FRWSA_TOPSIS):

- Steps implementing the MATLAB code FRWSA TOPSIS:

4. Plotting the **pie chart** using the Matlab *histcounts* command, which calculates the frequency of each alternative in the first row of the *RankMAT* matrix (corresponding to the TOP 1 of each ranking).
5. **Frequency distribution plot of the ranking position**. A *For* loop with the *histcounts* command calculates the frequencies for each of the rows in the *RankMAT* matrix (each row corresponds to the ranking position of each simulation).
6. Heatmap plot configuration.
   - *I.* *xlabel:* Title x-axis
   - *II.* *ylabel:* Title y-axis
   - *III.* *percentage_format = '%.2f%%'; h.CellLabelFormat = percentage_format*: result labels as percentages with two decimal

```
%% PLOTS
%% PIE PLOT:
figure(1)
pie3(histcounts(RankMAT(1,:), [1:N_Alt, inf]));
title('Best Alternative');
legend(Alt_Names);

%% HEATMAP
% Initialize frequency matrix
frec = zeros(N_Alt,N_Alt);

% Frequency of each number in each ranking
for j = 1:length(RankMAT(:, 1))
    row = RankMAT(j, :);
    frec(:, j) = histcounts(row, 1:N_Alt+1)./numCombinations.*100;
end

Rank = compose("Rank_%d", 1:N_Alt); % Labels for the x axis (Rank_x)

% Heatmap PLOT
figure(2)
h = heatmap(Rank, Alt_Names, frec,'Colormap',flipud(summer));
xlabel('Ranking');
ylabel('Alternative');
percentage_format = '%.2f%%';
h.CellLabelFormat = percentage_format;
```

User manual (FRWSA_TOPSIS):

- Steps implementing the MATLAB code FRWSA TOPSIS:

7. FRWSA plot

I. Get the first row of *RankMAT* matrix (TOP 1 of all rankings).

II. For loop to perform calculations for every alternative (Alt 1 → Alt k).

III. For each alternative, check the column where TOP 1 was to identify the winning weight combinations.

IV. Iterative process on each criterion to calculate the fraction in which each of the values that can have a weight appears in the winning combination.

V. Subplots configuration (as many subplots as criteria in the study.)

I. ylim: y-axis limit value

II. xsticks: x-axis labels (Correspond to the different weight values between 0-1).

```
%% FRWSA PLOT
Rank1row = RankMAT(1, :);                 % Top 1 Alternative in each TOPSIS simulation. E
AllWValues = 0:Incr:Incr*(Npoints-1); % All possible values that a weight can have.

for k = 1:N_Alt
    columnRank1 = Rank1row == k;          % Col looks at the Rank1row vector to see when
    WRank1Altk = WMAT(columnRank1, :);  % According to equation (13). WRank1Altx conta

    frc = zeros(Npoints, N_Crit); % Inizialisation of the 'f' matrix. This matrix will
    figsize=get(0, 'ScreenSize');
    figure(k+10);
    set(k+10,'Position',[figsize(1)+5 figsize(2)+50 figsize(3)-10 figsize(4)-135]);

    for j = 1:N_Crit
        for i = 1:length(AllWValues)
            Count = sum(abs(WRank1Altk(:, j) - AllWValues(i)) <= 0.0001);    % Calcula
            frc(i, j) = Count ./ repW(i);                                    % Accordi
        end

        % FRWSA Subplots
        subplot(ceil(N_Crit/2), 2, j);
        bar(AllWValues, frc(:, j));
        xlabel('Weights values');
        ylabel('\it frc');
        ylim([0 (Incr*(Npoints-1))])
        xticks(AllWValues)

        title(W_Names(j))
    end

    sgtitle(Alt_Names(k))
end
```

TOPSIS METHOD. CASE STUDY.

- For the sensitivity analysis case study, the same 5 car case study was used. The FRWSA sensitivity analysis evaluated the full range of combinations of weights for the 7 criteria. The weights were calculated for 21 different values (from 0 to 1 at 0.05 intervals). In total, 230,230 combinations were evaluated. This analysis assesses how often a car is the best with respect to the entire n-dimensional space of possible weight combinations.

CAR 1

- Trunk: 500 L
- MaxSp: 180 km/h
- Power: 306 hp
- L/100km: 1 L/100km
- Cost : 51,750 €
- N. Seats: 7
- Fuel : 2

CAR 2

- Trunk: 510 L
- MaxSp: 180 km/h
- Power: 110 hp
- L/100km: 4.8 L/100km
- Cost : 24,540 €
- N. Seats: 5
- Fuel : 5

CAR 3

- Trunk: 470 L
- MaxSp: 160 km/h
- Power: 220 hp
- L/100km: 0 L/100km
- Cost : 43,488 €
- N. Seats: 5
- Fuel : 1

CAR 4

- Trunk: 395 L
- MaxSp: 196 km/h
- Power: 120 hp
- L/100km: 5.5 L/100km
- Cost : 29,800 €
- N. Seats: 4
- Fuel: 4

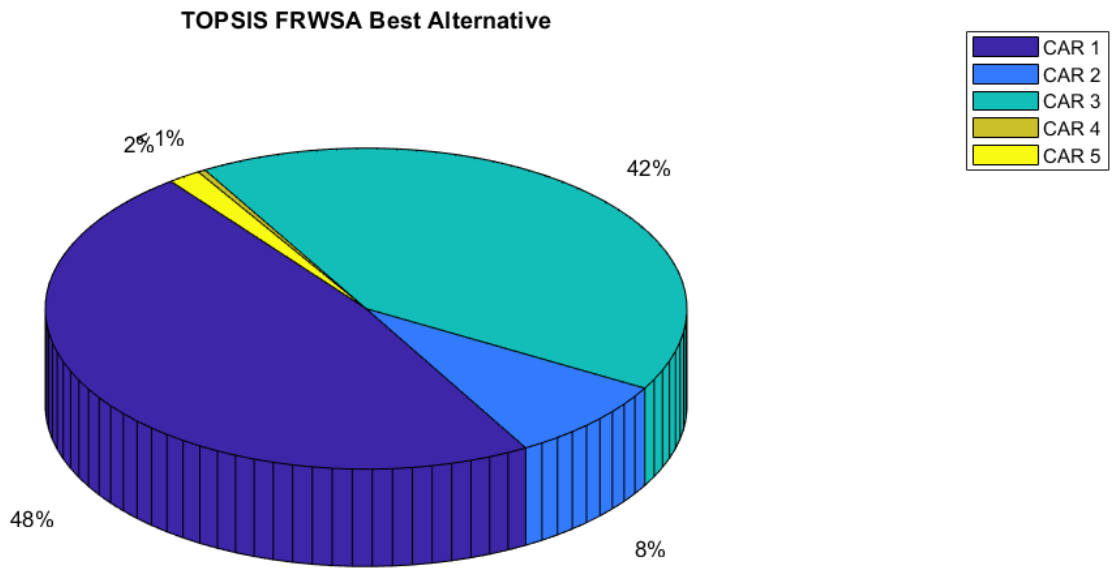
CAR 5

- Trunk: 377 L
- MaxSp: 170 km/h
- Power: 122 hp
- L/100km: 5.3 L/100km
- Cost : 29,700€
- N. Seats: 5
- Fuel: 3

SA. CASE STUDY.

– Pie Plot

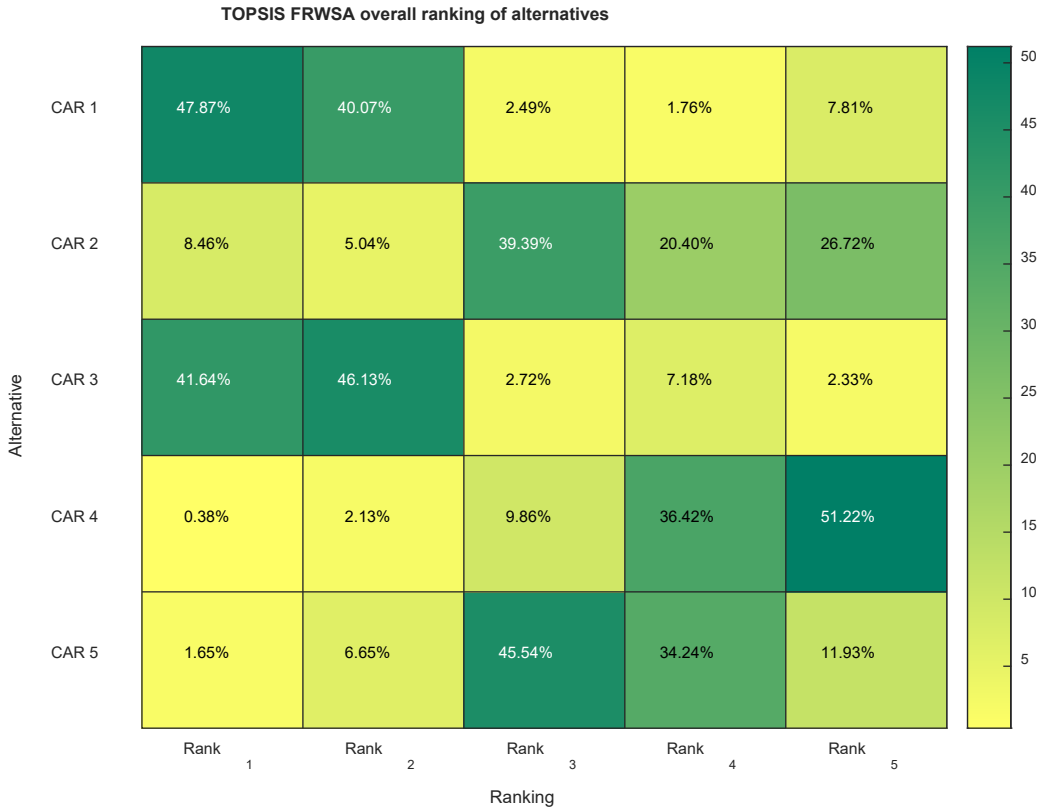
- The pie chart shows the percentage distribution of the top-ranked alternative.
- Car 1 shows a dominance over the others, being the best ranked in 48% of the cases. This alternative obtains the best results compared to the others for most of the criteria.
- Car 3 is the second best (42%).
- Car 2 the third best (8%).
- Car 5 the fourth best with only 2%
- Finally, Car 4 is the worst performer (< 1%), with the lowest frequency and the worst results for most of the criteria.



SA. CASE STUDY.

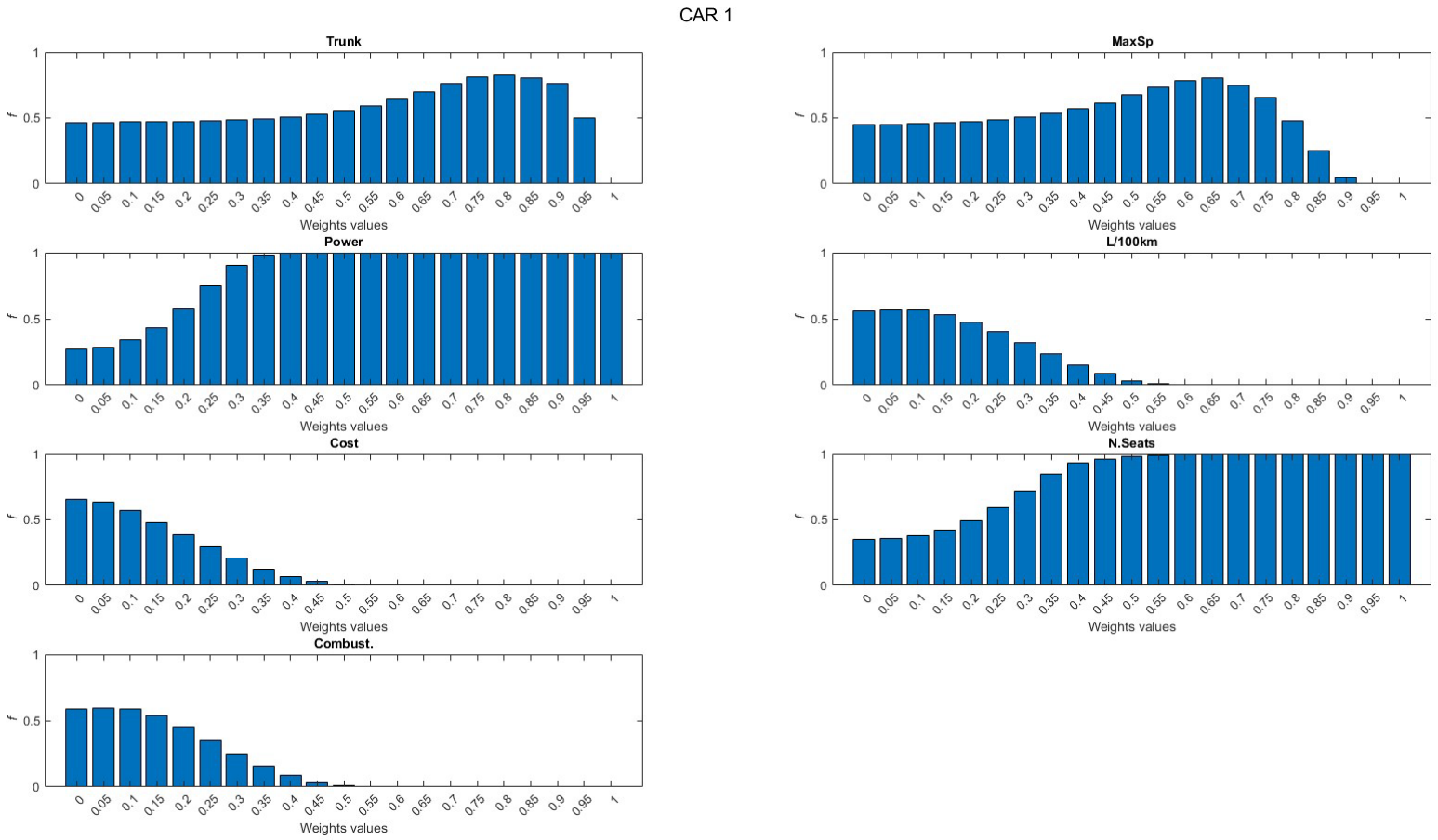
– Frequency distribution plot of the ranking position

- Car 1 and Car 3 have a strong presence in the first position (47.87% and 41.64% respectively), often secure the second position and are the least frequent in the last position, demonstrating their superior performance.
- The next most successful alternative is Car 2 (8.46%), but it is also often the worst alternative (26.72%).
- On the other hand, Car 4 and Car 5 are the lowest ranked alternatives and consistently appear as the worst choice (0.38% and 1.65% respectively), with Car 4 being the alternative that appears most frequently in the last position (51.22%).



SA. CASE STUDY.

- **FRWSA Plot CAR 1**



Car 1 is the best alternative for a significant fraction of the weight alternatives.

Focusing on the trunk criterion, in the 0-0.95 weight range, it is consistently the best alternative 50% or more of the time.

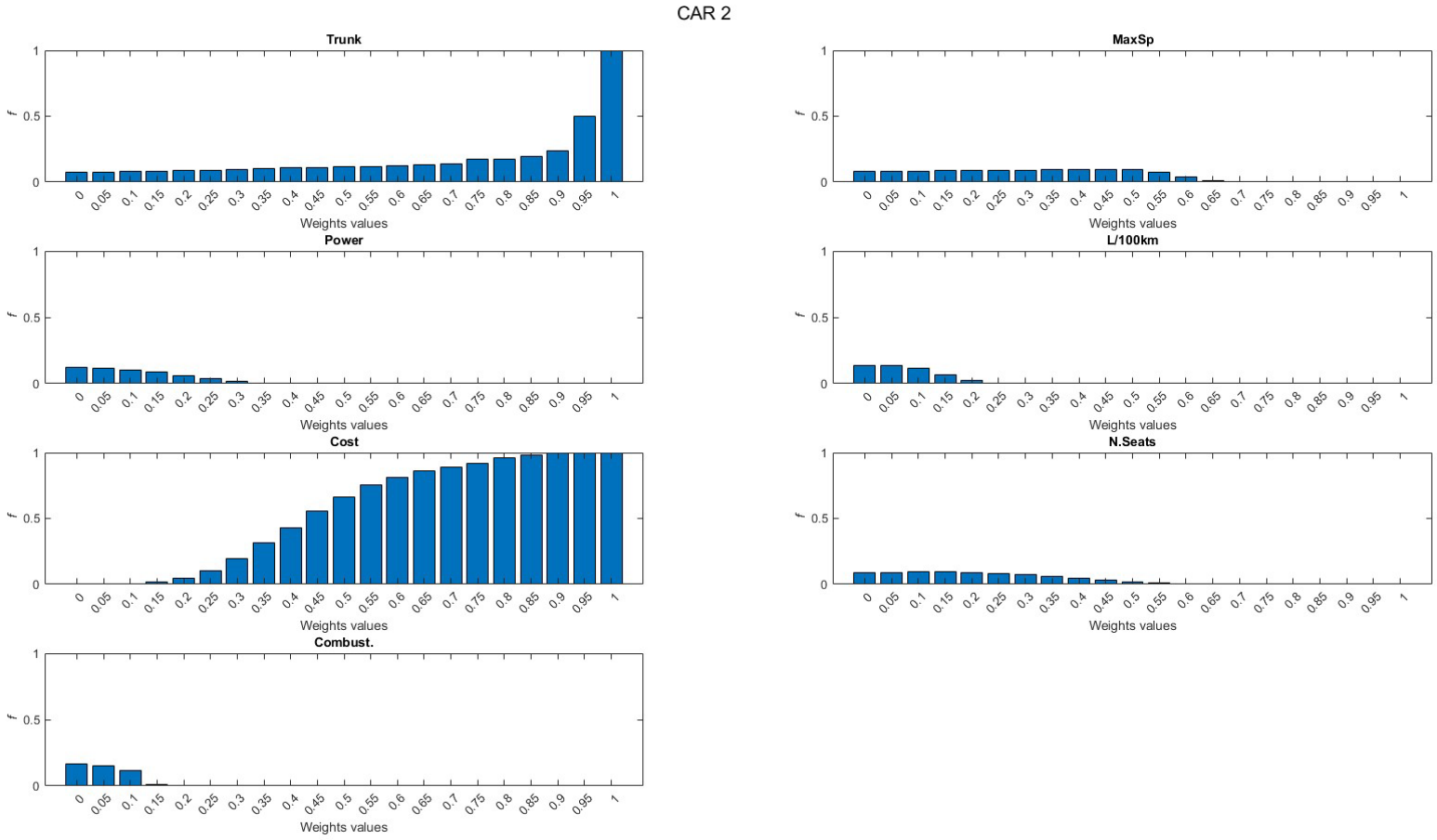
It is always the best alternative when the weight of Power is greater than 0.35 or the number of seats has a weight greater than 0.6.

For MaxSp it is always one of the best alternatives.

In terms of Cost, L/100km and Combustion, it must have a low weight value to be the best option, which means that these criteria do not favour this alternative.

SA. CASE STUDY.

- **FRWSA Plot CAR 2**



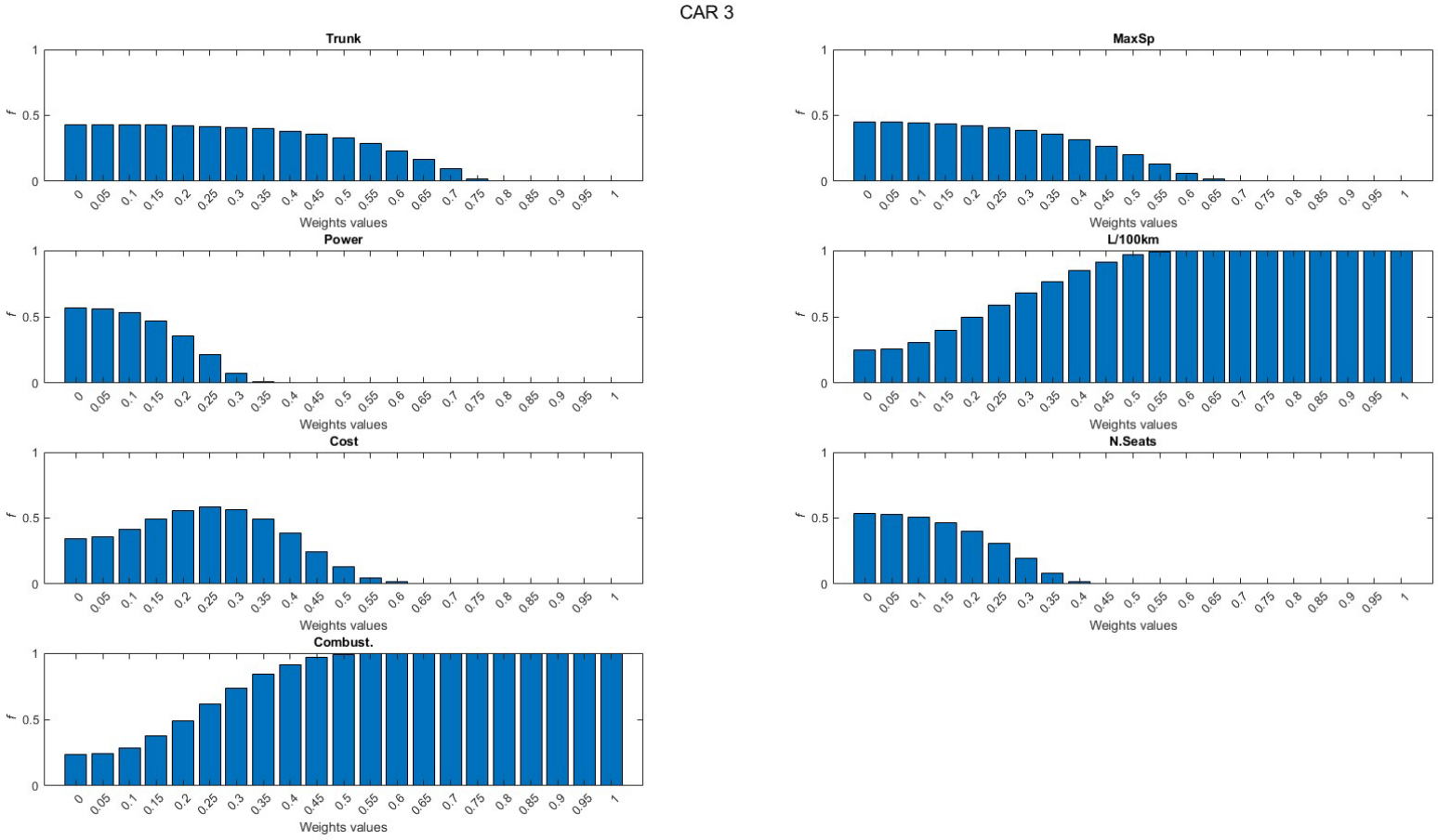
Car 2 is a good alternative if cost is taken into account. If the cost weight is higher than 0.45, it becomes the best alternative.

If trunk is considered as the only criterion, it is recognised as the best alternative because it is the best in this criterion. As this weight decreases, it becomes a bad alternative in most cases.

It does not become a good option when other criteria are considered.

SA. CASE STUDY.

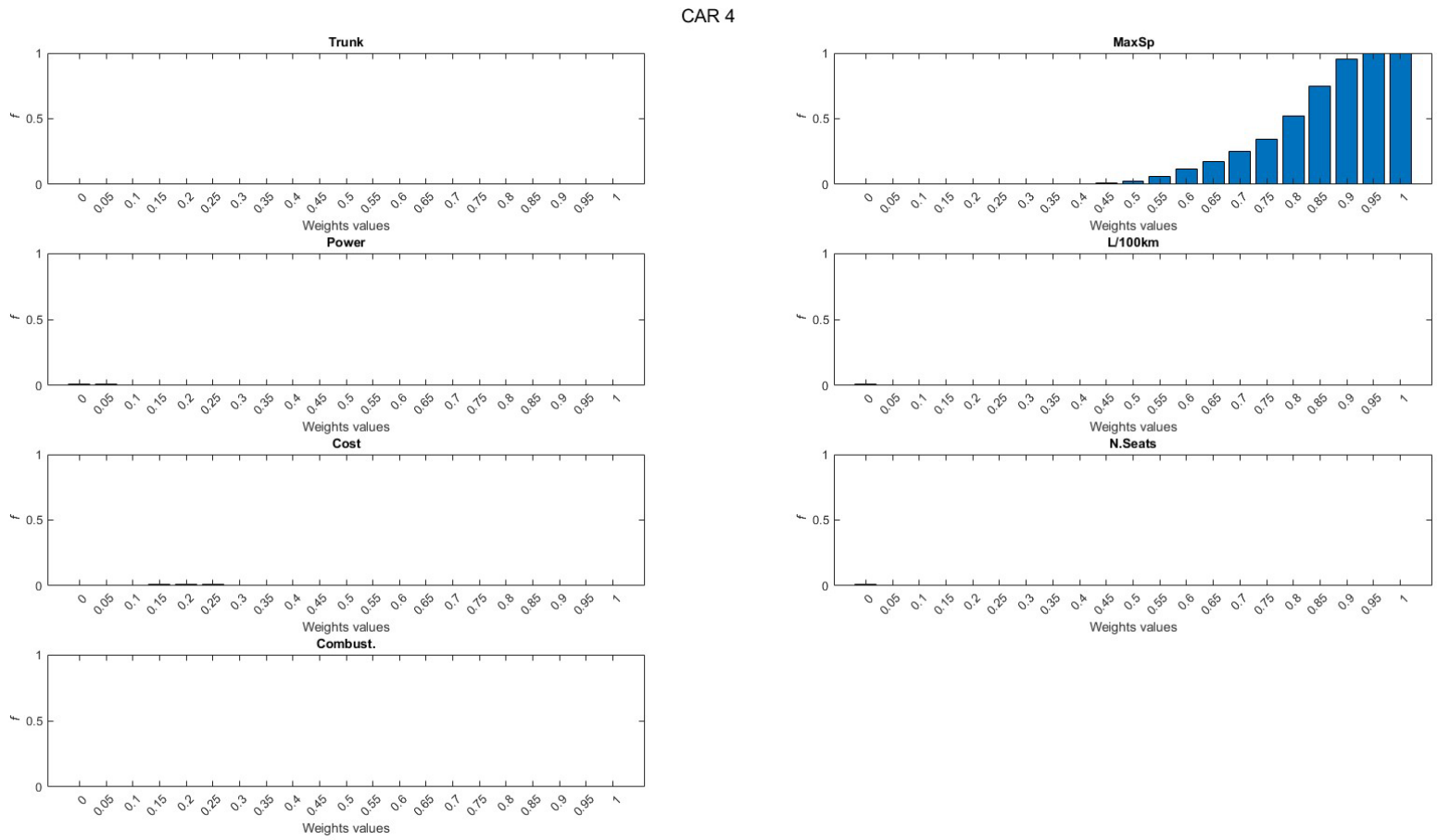
- **FRWSA Plot CAR 3**



Car 3 is also a good alternative, similar to Car 1.
It turns out to be a very good alternative when considering the criteria L/100km and Combustible, being almost the best option when these weights are higher than 0.5.
It is also a good alternative in some other cases when considering Trunk, MaxSp, Cost Power and N.Seats for a good range of weights.

SA. CASE STUDY.

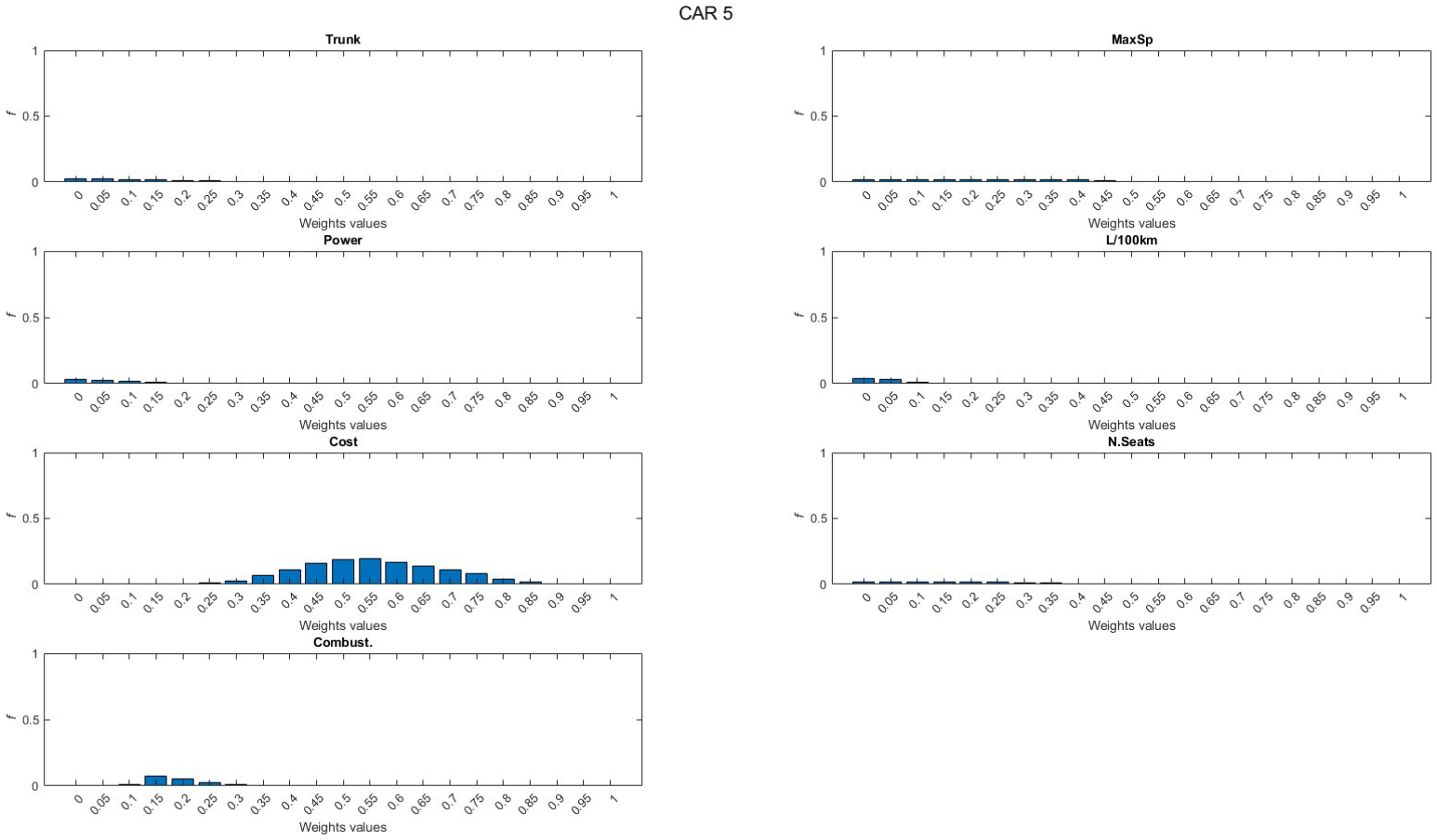
- **FRWSA Plot CAR 4**



Car 4 is the second worst option, and it only makes sense if MaxSp is very important, a weight higher than 0.8 gives more than 50% of cases where Car 4 is the best.
In all other cases it is not the best option.

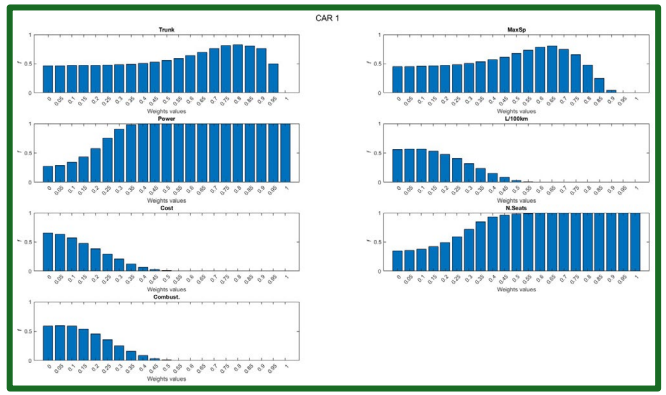
SA. CASE STUDY.

- **FRWSA Plot CAR 5**

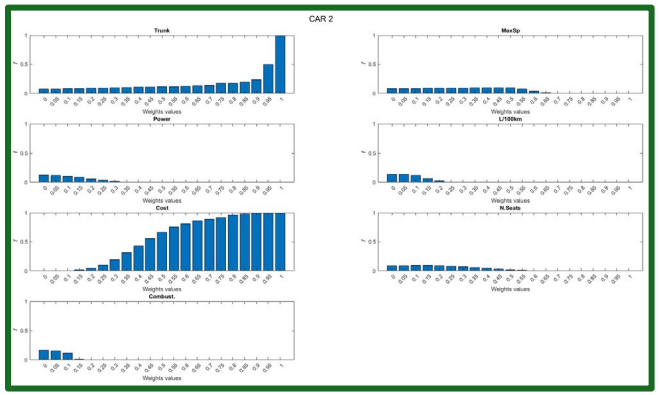


Finally, Car 5 is the worst option and is the best option only in a very small fraction of cases with intermediate weights for the cost criterion.
In all other cases it is never the best option.

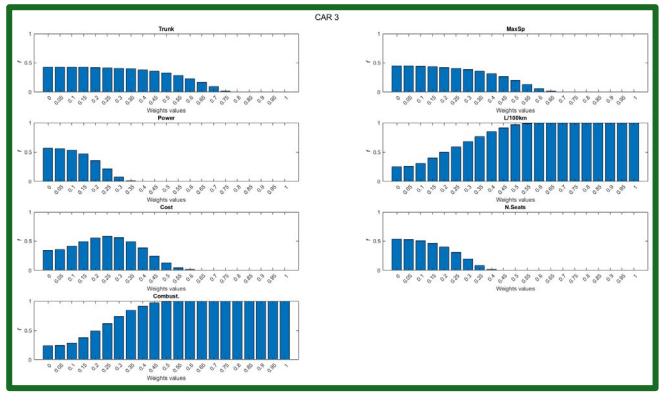
CAR 1



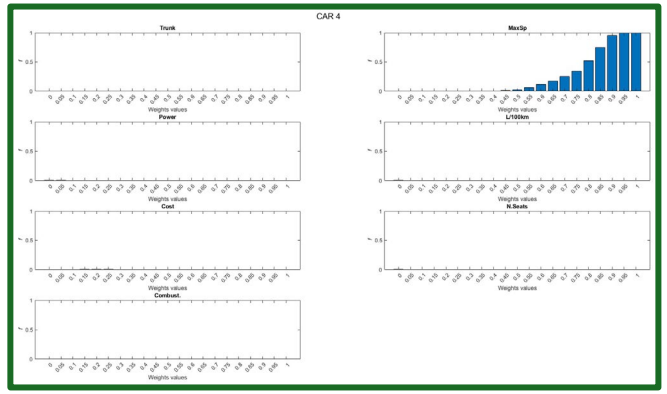
CAR 2



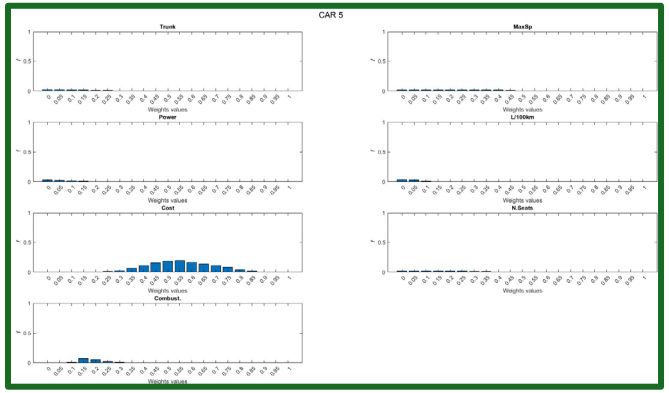
CAR 3



CAR 4



CAR 5



Comparing the distributions of all cars simultaneously for each criterion, the sum of all car alternatives always equals 1, and we can see the distribution of the fraction of cases where each car is better.

These figures also show how, when only one criterion is considered (weight value 1), the alternative with the best value for this criterion is shown with a fraction of 1 in the corresponding figure. It also shows whether this best value is only marginally interesting (as in the case of Car 2 and the trunk), because as soon as the weight decreases a little it is no longer an interesting alternative, or whether, on the contrary, it has a large weight, as in the case of Car 1 and the power criterion, where this criterion determines the best option as long as it has a weight greater than 0.35.

Conclusions Sensitivity Analysis TOPSIS method:

- The WSA method provides information on how much a weighting can increase or decrease without changing the outcome of the highest ranked alternative and reveals which weights are most sensitive.

- The new FRWSA sensitivity analysis presented in this study greatly improves our understanding of the dynamics of the weights within the TOPSIS method.

- The FRWSA method provides the n-dimensional study of a full range of all possible combinations of weight values, providing comprehensive information and showing the strengths and weaknesses of each alternative.

- This study not only improves the accuracy and efficiency of decision making when selecting an alternative in engineering, but also provides a deeper understanding of the underlying mechanisms of the method and can be applied to all fields where multi-criteria analysis is required.

AN INTEGRATED TOPSIS FRAMEWORK WITH FULL-RANGE WEIGHT SENSITIVITY ANALYSIS FOR ROBUST DECISION ANALYSIS

UAB
Universitat Autònoma de Barcelona

Research Group on Biological Treatment and Valorisation of Liquid and Gaseous Effluents
GENOCOV

Àlex Gaona Soler, Albert Guisasola and Juan Antonio Baeza

Departament d'Enginyeria Química, Biològica i Ambiental
Escola d'Enginyeria. Universitat Autònoma de Barcelona

Email: alex.gaona@uab.cat, albert.guisasola@uab.cat, juanantonio.baeza@uab.cat